Cross-Robot Behavior Adaptation through Intention Alignment

Xi Chen[1†*], Yuan Gao[3,2†], Hangxin Liu[1,4], Fangkai Yang[5], Ali Ghadirzadeh[6], Jun Yang[7], Bin Liang[7], Chongjie Zhang[8*], Tin Lun Lam[2,3*], Song-Chun Zhu[1,4,7]

[1]State Key Laboratory of General Artificial Intelligence,
Beijing Institute for General Artificial Intelligence (BIGAI), Beijing, China
[2]School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen, China
[3]Shenzhen Institute of Artificial Intelligence and Robotics for Society, Shenzhen, China
[4]School of Intelligence Science and Technology, Peking University, Beijing, China
[5]Microsoft, Beijing, China
[6]Embark Studios, Stockholm, Sweden
[7]Department of Automation, Tsinghua University, Beijing, China
[8]McKelvey School of Engineering, Washington University in St. Louis

[†] These authors contributed equally to this work and are listed alphabetically
[*] Corresponding authors

This is the Author Accepted Manuscript (AAM) of the article published in *Science Robotics*. The final published version is available at https://doi.org/10.1126/scirobotics.adv2250.

Imitation learning (IL) has succeeded in enabling robots to perform new tasks by learning from demonstrations. However, its success is often constrained by the need for direct skill mappings between a learner and a demonstrator under identical conditions, limiting its adaptability to diverse environments and generalization across robots with different physical embodiments. To address these challenges, we introduce the Intention-Aligned Imitation Learning (IAIL) framework, a behavior adaptation approach that extends the conventional scope of IL by enabling robots to reproduce motions demonstrated by heterogeneous peers, even in novel situations. Inspired by human cultural learning, IAIL aligns and adapts robot motions based on high-level intentions annotated in natural language, rather than by directly copying motor movements. This alignment is achieved by constructing a shared intention space that connects robot-generated motions with linguistic annotations, enabling inference-time behavior adaptation across diverse embodiments and environmental contexts. The framework further supports scalable task allocation within heterogeneous robot teams by leveraging differences in capabilities and constraints. We validate IAIL through real-world experiments involving seven distinct robots performing multi-step collaboration tasks across 30 scenarios. Our results demonstrate that IAIL enables robust intention-aligned behavior adaptation across variations in embodiment, motion modality, and task configuration. These capabilities enable flexible behavior transfer across heterogeneous robots and support resilient, autonomous multi-robot systems for reliable real-world collaboration.

INTRODUCTION

Deploying a team of robots to perform long-horizon tasks in dynamic real-world environments holds substantial potential to enhance productivity in manufacturing, increase system resilience in adversarial conditions, and enable collaborative behaviors beyond the capabilities of individual robots. However, programming a robot team with proficient individual skills and efficient task allocation could be difficult, particularly when the team varies substantially. Imitation learning (IL) has emerged as a promising method for enabling robots to efficiently acquire new skills by learning from expert demonstrations (*1, 2, 3*), thereby facilitating skill transfer for robotic systems. However, most IL approaches necessitate that the learner and demonstrator operate under identical task conditions and have precise mappings between their motor actions, limiting the adaptability of the tasks demonstrated in different environments or generalizability in learning from robots with different physical embodiments.

The primary objective of IL is to establish meaningful motion correspondence between the actions of the demonstrator and the learner, thereby ensuring comparable outcomes after execution. However, achieving this correspondence becomes increasingly challenging in the presence of variations in environmental conditions and robotic embodiments (*4, 5, 6*). A comparison of IL settings with variations between the demonstrator and learner is illustrated in Fig. 1. For differences in physical embodiments, such as robots with different degrees of freedom, body structures, or actuator types, motion correspondence can be established based on invariant body components (*7, 8*) or transitions in environmental states (*9, 10*). For environmental variations, such as different lighting conditions, varied camera views, or interactable objects with changing colors or textures, motion correspondence can be established by finding a common feature space with domain confusion approaches (*11, 12*). Although these strategies have shown success under moderate variations, they become inadequate in cases involving substantial differences, such as between robots with fundamentally different motion modalities (ground vehicles versus drones) or environments with distinct sets of interactable objects (office versus home). In such settings, direct correspondence becomes unreliable due to mismatched robot capabilities and functionalities. To address this issue, recent works have suggested learning correspondence based on completed tasks or the final motion outcomes (*13, 14, 15, 16*). However, these methods require datasets with manually labeled or paired motion trajectories for each learner-demonstrator combination. This pairing process demands extensive engineering effort, which limits the scalability of these methods to IL scenarios involving an increased number of robots. Alternatively, some methods leverage unsupervised learning techniques to learn the correspondence (*17, 18, 19*), which eliminates the engineering effort but requires robots with identical functionalities. An efficient approach that can be adapted to various environmental conditions and generalized to diverse robot forms is still lacking.

Moving beyond the agent-to-agent imitation setup, enabling a team of robots to replicate the

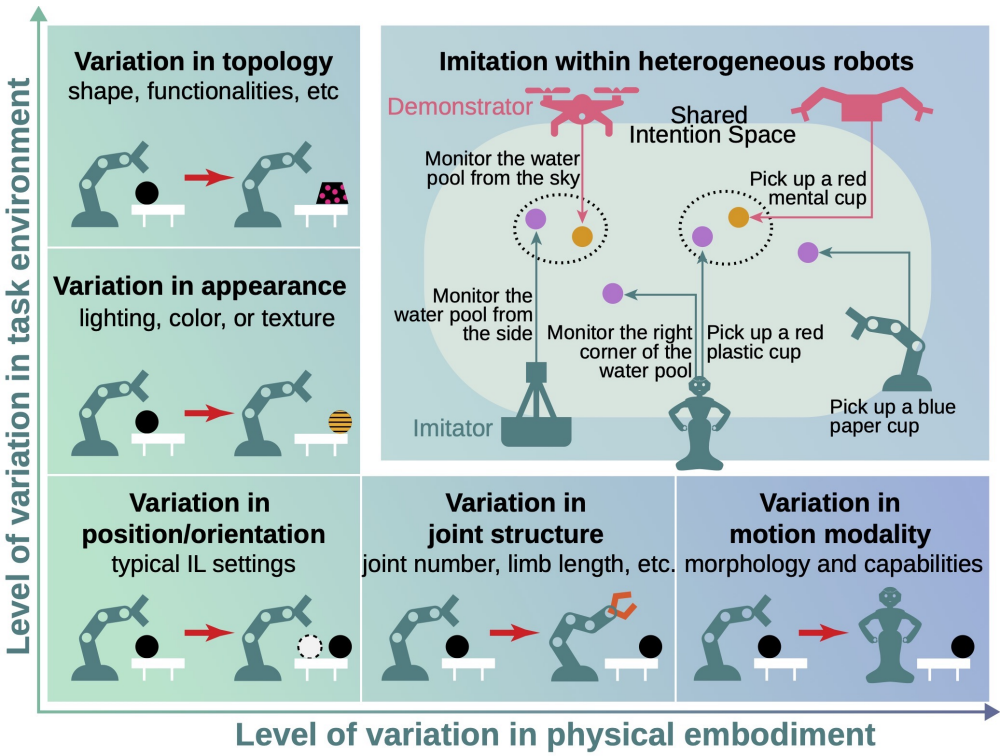

Figure 1: **A visual illustration of different imitation learning scenarios.** In prior works, imitation learning typically occurred between two robots with varying degrees of differences in physical embodiments or task-specific environmental conditions. In contrast, our work explores a setting where heterogeneous robot teams, characterized by substantial differences in both embodiments and environmental conditions, can imitate each other through motion demonstrations. This imitation is achieved by associating motions based on their underlying intentions, which are defined using human-annotated language descriptions. The association is performed within a shared intention space, constructed by aligning the embeddings of motion sequences with their corresponding language intention annotations. Each motion of the demonstrator robot is matched to the motion of the learner robot that has the smallest distance in the intention space. The dashed circles in the figure indicate successful motion associations within the intention space.

multi-step collaborative tasks performed by another team introduces a team-to-team imitation setup. When considering heterogeneity in team size, robot types, and individual robot capabilities, the critical challenge lies in enabling the learned model to generate feasible motion plans for the learner robot team, which may differ from the demonstrations; and to correctly assign each motion step to the appropriate learner robot based on its capabilities. This necessitates an integration of IL for multi-robot teams (*20, 21, 22, 23*) and multi-robot task allocation (MRTA) (*24, 25*). However, owing to the varied functionalities of the learner and demonstrator robots and the unclear task definitions represented by implicit motion trajectories, imitation between heterogeneous robot teams remains

an unexplored topic.

To transcend these limitations and enable imitation across robot embodiments and task conditions, we propose aligning robot behaviors based on shared high-level intentions. Rather than replicating low-level motor actions or embodiment-specific features, our approach compares and associates behaviors according to their underlying task objectives, supporting both agent-to-agent and team-to-team transfer. We define an intention as the goal or outcome of a motion and represent it using a human-annotated language description that abstracts away control and embodiment details. We then construct a shared intention space by aligning motion embeddings with their corresponding annotations, enabling intention similarity to be measured across heterogeneous robots. At inference time, a learner retrieves the most demonstration-relevant behavior from its pre-trained repertoire by matching intentions. This formulation extends naturally to team-level settings: given multiple learners, the system assigns each demonstrated step to the robot that can most feasibly realize the intended outcome within its repertoire, enabling capability-aware task allocation across varying team compositions. We refer to this framework as Intention-Aligned Imitation Learning (IAIL).

This approach draws inspiration from human cultural learning mechanisms, where individuals understand others' actions in terms of intentions and re-express them through contextually suitable behavior. This aligns with the cognitive science literature on rational imitation, which suggests that learners, even infants, prioritize reproducing a demonstrator's inferred goals over their exact movement patterns (*26, 27*). Studies in neuroscience further support this view, indicating that humans interpret behavior at an intentional level rather than via motor mimicry (*28, 29, 30, 31, 32, 33, 34, 35*).

In this article, we study imitation across a diverse set of robots with distinct embodiments, operating environments, and capabilities, where any individual robot or team of robots may act as a demonstrator or a learner. We evaluate our framework on seven real-world robots, including boats, drones, mobile robots, and robotic arms, performing a compound task spanning multiple phases. The task is demonstrated by a team of three robots and executed by a separate team of three or four robots, with no overlap between demonstrators and learners. Across 30 scenarios with varying task and team configurations, our framework achieves robust task completion across diverse embodiments and motion modalities. These results demonstrate generalizability across robots, adaptability to varying tasks and environmental conditions, and scalability to heterogeneous teams with different compositions. We further compare IAIL against two baseline methods in simulation, quantifying consistent gains under multiple sources of variation. Together, these results highlight the potential of intention-aligned imitation to enable more flexible and efficient learning in multi-robot systems and to extend imitation learning to dynamic real-world deployments.

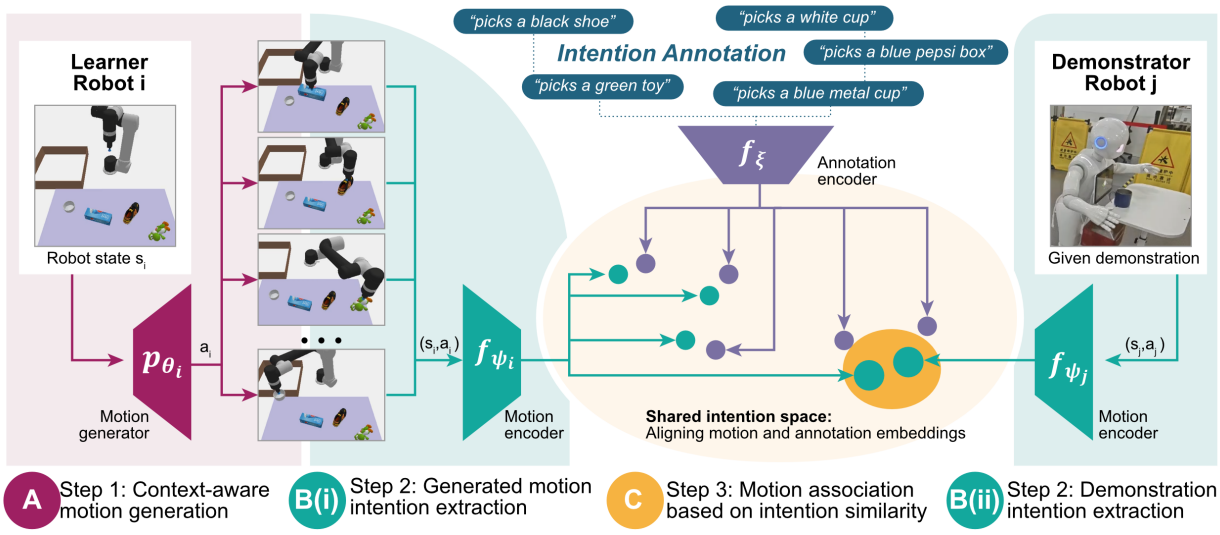

Figure 2: **Overview of the IAIL framework in an agent-to-agent scenario.** The diagram illustrates the process of learner robot i reproducing an action demonstrated by robot j. The process involves three main stages: **(A)** A batch of executable actions, each capable of achieving various goals, is sampled from the pre-trained motion generator p_{θ_i} of robot i. **(B)**(i) The intentions of the sampled motions are extracted by projecting them into a shared embedding space via the motion encoder f_{ψ_i} of robot i. (ii) In parallel, the intention of the demonstrator's motion is extracted using the motion encoder f_{ψ_j} of robot j. The shared embedding space is regularized by the annotation encoder f_ξ, trained jointly with the motion encoders to align motion embeddings with their high-level intentions. **(C)** The demonstration is then associated with the sampled learner motion whose embedding is closest in the shared intention space. Through this generation–encoding–association process, the learner interprets the demonstration and adapts it into an executable action aligned with the demonstrated objective.

RESULTS

We evaluated our IAIL framework through experiments involving heterogeneous robots performing IL tasks in dynamic environments. The framework associates behaviors through underlying intentions rather than direct motor correspondence. This design enables adaptation across embodiments and environmental contexts and preserves the objectives of the demonstrations. This section presents an overview of our framework, followed by the experimental validation, detailed results, and a comparative analysis of our approach with baseline methods.

IAIL framework overview

Fig. 2 provides an overview of the IAIL framework in an agent-to-agent scenario, illustrating the core modules, the three main stages of the imitation process, and how these modules interact across stages. The figure is intended as a high-level guide to the system's structure; detailed descriptions of each module and stage are provided in the Materials and Methods section.

The framework enables imitation among heterogeneous robots by associating actions based on shared intentions. The process consists of three key stages: context-aware motion generation, motion intention extraction, and motion association based on intention similarity. Below, we describe each stage, identifying the networks involved, their training, and their role in the overall process. A video demonstration of the three stages is included in the Supplementary Materials Movie S2.

Context-aware motion generation

In this stage (Fig. 2A), we generate feasible actions for the learner robot i based on its current state, defined as the information perceived by its onboard sensors, which captures its operational context within the environment. Each action is a single command or a sequence of commands that the learner can execute to achieve a specific goal in its environment. The set of generated actions reflects the current capabilities or skills of the learner robot in its present context and serves as candidates for intention-based association in later stages. These actions are sampled from a generative model p_{θ_i}, trained offline using a dataset collected from robot i.

Motion intention extraction

In this stage (Fig. 2B(i-ii)), both the learner's generated actions and the demonstrator's executed actions are projected into a shared motion–intention embedding space. The projection is performed using a motion encoder $f\psi_i$ for the learner and $f\psi_j$ for the demonstrator, each trained jointly with a shared annotation encoder f_ξ. Training follows a contrastive learning objective in which human-annotated language descriptions of motions provide semantic supervision. This ensures that actions with similar annotated intentions, regardless of embodiment, are embedded close to each other in the space. Through this process, the system obtains an intention-level representation for both the demonstration and each candidate action, enabling embodiment-independent comparison.

Motion association based on intention similarity

In the final stage (Fig. 2C), we compare the embedded representation of the demonstrator's motion with the embeddings of the learner's candidate actions generated in the first stage. The system selects the candidate whose embedding is closest to the demonstrator's embedding in intention space, ensuring that the chosen action is executable for the learner and aligned with the demonstrated

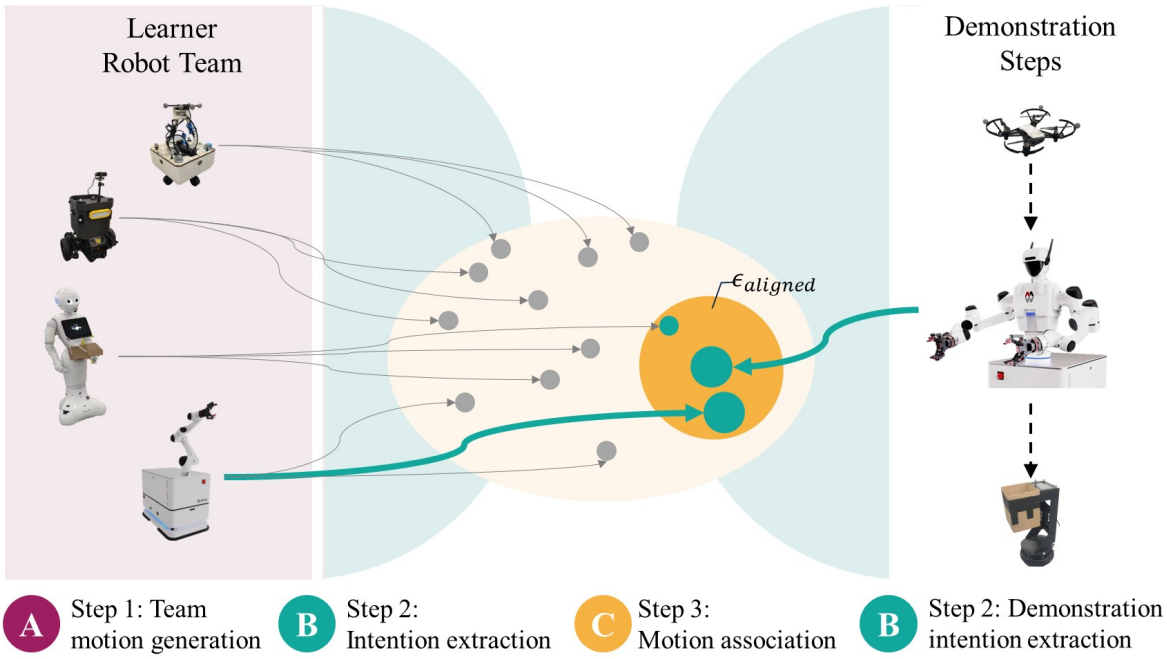

Figure 3: **The process of action association between robot teams**. We process the demonstrations provided by the demonstrator team individually. For each demonstration, we use the following steps: (**A**) First, a candidate batch of actions from all the robots on the learner team is sampled using their motion generators given their current contexts. (**B**) Next, we extract the intentions of these actions by projecting them to a shared embedding space using their motion encoders. (**C**) Finally, we associate the demonstration with one of the sampled actions that is within the boundary of $\epsilon_{aligned}$ and is closest to the demonstration in the embedding space. The selected action is then sent to the robot that generated this action for execution.

objective. When multiple learner robots are available, we perform this selection over all candidates from all learners and assign the best-matching action to the robot that generated it. The team-level motion association process is shown in Fig. 3.

Team-to-team imitation setup

We evaluated the IAIL framework in a complex real-world environment involving seven heterogeneous robots performing multi-step tasks. The robots had distinct capabilities and operated in a shared environment comprising several areas, each associated with different types of tasks. The tasks included: monitoring user activities at one of four locations (M_{1-4}), fetching items at one of three areas (I_{1-3}), and delivering items to one of two destinations (D_{1-2}). A visual illustration of the robots, environment, and related tasks is shown in Fig. 4A.

Seven robots were involved in the study and were divided into two teams: the demonstrator

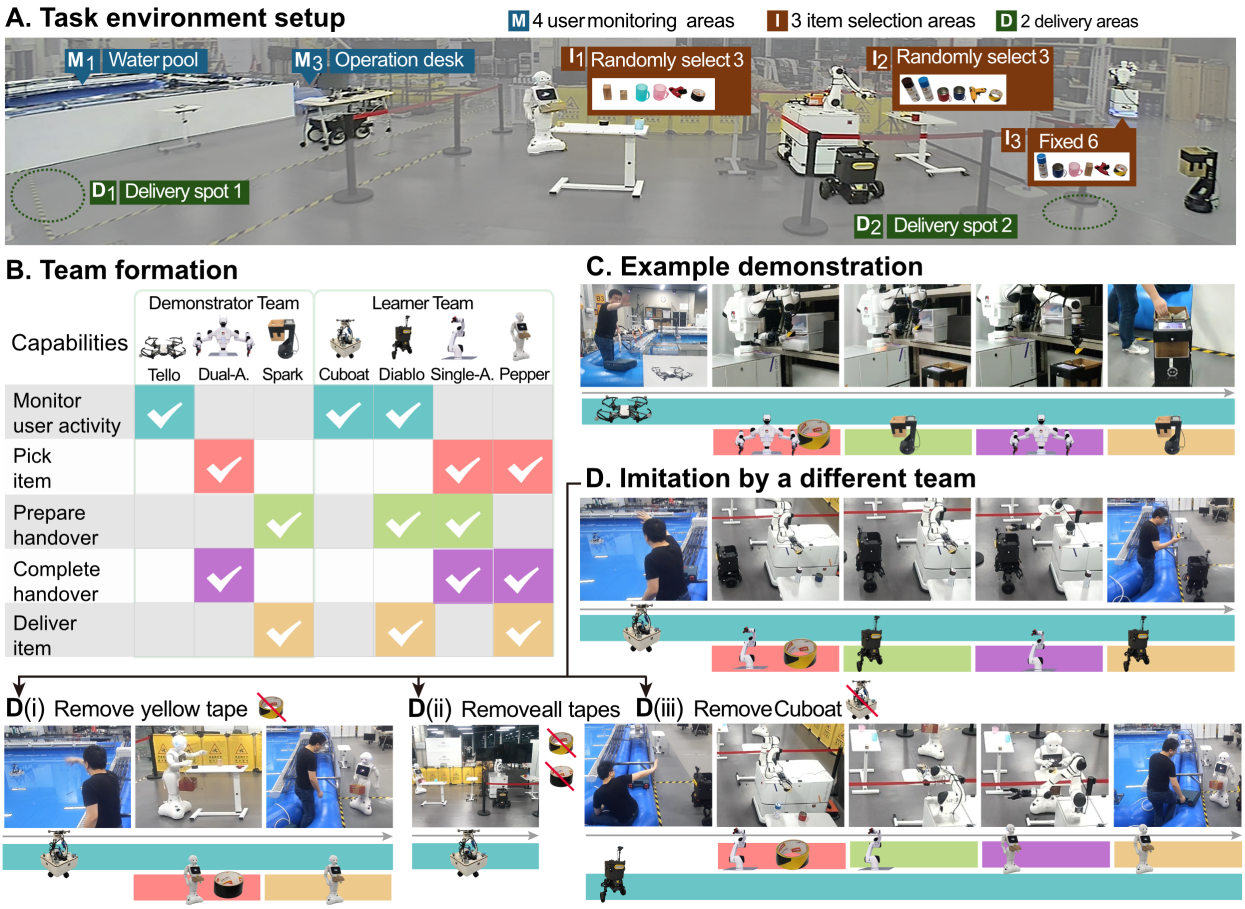

Figure 4: **The real-world experiment setup and imitation results obtained under various conditions.** **(A)** The imitation task has three phases: monitoring user activity in M_{1-4}, fetching an item from I_{1-3}, and delivering it to D_1 or D_2. **(B)** Given that the robots on the learner team are different from those on the demonstrator team, direct imitation is not possible. **(C)** An example 5-step demonstration performed by Tello, Dual-Arm, and Spark: Tello monitors the user signal, Dual-Arm picks a yellow tape, Spark moves closer to initiate an item handover, Single-Arm passes the item to complete the handover, and Spark delivers the item. **(D)** The imitation performed by the learner team. Our IAIL framework successfully allocate the imitative actions to the corresponding robots based on their capabilities. (i) When the yellow tape is removed, the black tape is identified as an alternative, and the Pepper robot is assigned to pick up and deliver it. When neither the yellow tape nor the black tape is available, the team remained idle, as no imitative actions that could fulfill the intention were possible as shown in (ii). In (iii), Diablo was selected to monitor the user when Cuboat was removed from the team, which subsequently changed the imitative actions of the other robots. In the various cases, the executed performance differs substantially from the demonstration.

team consisted of Tello (*36*), Dual-Arm (*37*), and Spark (*38*), whereas the learner team consisted of Cuboat (*39*), Single-Arm (*37*), Pepper (*40*), and Diablo (*41*) (see Fig. B). Because of embodiment

and workspace constraints, each robot had access to a subset of areas and supported distinct capabilities. Tello was a drone that can monitor all four locations $M_{1\text{–}4}$. Dual-Arm had two manipulators and can retrieve items from the drawer at I_3, whereas Single-Arm can retrieve items placed on the table at I_2. Spark was a mobile transporter that can deliver items to D_1 and D_2. Cuboat operated in the water pool and can monitor locations M_1 and M_2. Diablo was a height-adjustable mobile robot that can monitor all four locations $M_{1\text{–}4}$ and deliver items to D_1 and D_2. Pepper was a mobile manipulator that can access items on the table at I_1 and also deliver items to D_1 and D_2. The detailed robot and environmental setup is provided in the Supplementary Materials (Section Real-world experiment setup).

During the IL process, the demonstrator team provided a demonstration consisting of five actions. First, Tello flew to one of the four monitoring locations (M_1 to M_4) to monitor user activities. The user then entered the area and signaled the robot by waving at its camera. Once the signal was detected, Dual-Arm picked an item from the drawer at I_3 and passed it to Spark for delivery. This handover process involved two actions: Spark initiated the handover by moving closer to Dual-Arm, and then Dual-Arm placed the picked item into Spark's basket to complete the handover. Finally, Spark delivered the item to one of the delivery locations (D_1 or D_2) to complete the task. An example demonstration given in Fig. 4C shows that the when the user detected pool leakage and sent signals, the team of robots monitored this activity at M_1, fetched a yellow tape, and delivered the tape to D_1.

Following the five-step demonstration, the learner team was required to replicate the final outcome: delivering the correct item to the appropriate location. However, the user's location, the item type, and the delivery destination were not explicitly provided, requiring the team to infer this information from the demonstration.

Team-to-team imitation results

To assess the effectiveness of the proposed IAIL framework, we evaluated performance from two perspectives. First, we measured task success rate and adaptation accuracy, which reflected how well the learner robots imitated the demonstrated behaviors and achieved the intended outcomes. Second, we analyzed how tasks were executed across varying environmental configurations and team compositions, highlighting the system's flexibility, robustness, and adaptability under dynamic conditions. We began by introducing the evaluation scenarios, followed by detailed results under these two perspectives.

Evaluation Scenarios

To assess the system's ability to handle variations in environmental conditions and team formations, we conducted 30 scenarios with varying task configurations for both the demonstration and the

learner robot team. In each scenario, the demonstration involved a unique user location, selected from M_1 to M_4, an item, chosen from the six items in I_3, and a delivery position, selected from D_1 or D_2. For the learner robots, the three items at I_1 and I_2 were randomized from their respective lists in each scenario. Additionally, in one-third of the scenarios, one robot was randomly removed from the learner team, leaving only three robots to complete the tasks. The variations between the training and evaluation scenarios were provided in the Supplementary Materials (Section Training and evaluation data variations).

Due to these random variations in robot availability and item distribution, six of the 30 scenarios were completion infeasible, either because no robot possessed the necessary capabilities or because the relevant item from the demonstration was entirely unavailable. Among the 24 feasible scenarios, 11 included the exact demonstrated item (same item available), whereas the remaining 13 included a functionally equivalent item but with a different shape, color, or specific attribute from the same semantic class (same-class item available). The item class is shown in fig. S3. To ensure the reliability and statistical significance of our results, we repeated the evaluation three times on the same 30 scenarios, using models trained with different random seeds.

Task Success Rate

A task was considered successful when the exact demonstrated item, or an item within the same class, was delivered to the correct location. The task success rate was evaluated as the proportion of successful trials out of the total number of scenarios that could be completed successfully. This metric captured the overall effectiveness of IAIL in consistently imitating behaviors across teams of heterogeneous robots.

Table 1: **Average performance over 30 diverse evaluation scenarios ± one standard deviation.** Each value reports the average number of successful or best-adapted scenarios out of the total number of applicable cases. The evaluations were repeated three times with different random seeds.

	Num. of Scenarios	**Mean Achieved Trials**	**Success Rate**
Task Success	24	22 ± 1.00	92%
Best Adaptation	30	26.33 ± 1.15	88%
Same item available	11	9.33 ± 0.58	85%
Same-class item available	13	11.33 ± 1.15	87%
Completion infeasible	6	5.67 ± 0.58	94%

As shown in Table 1, the robots successfully completed an average of 22 out of 24 scenarios, achieving an overall success rate of 0.92. This high success rate demonstrated the system's ability to effectively enable imitation between heterogeneous robot teams, allowing them to execute multi-step tasks through action demonstrations.

There were seven failures among the 30 scenarios: one involved the robot picking an irrelevant item, whereas the other six remained inactive even when the exact item or an item of the same class was presented. These failures occurred primarily due to incorrect extraction of demonstration intentions, which could result from factors such as model errors or sensory noise. When this occurred, the intention distances between the demonstration and the sampled actions become large, and the sampled actions were identified as incapable of replicating the demonstration. In such scenarios, the robot opted to remain inactive rather than risk performing an action that could lead to an unexpected or undesirable state. This conservative behavior was critical in real-world applications where safety was paramount.

Adaptation accuracy

In addition to the task success rate, we evaluated the robots' ability to adjust their behavior under different environmental conditions using the adaptation accuracy metric. This metric measured the percentage of instances in which the robots achieved the best possible outcome under three conditions. If the demonstrated object was available, the learner robots were expected to deliver that object. If it was unavailable but a same-class object was available, the system was expected to deliver the available object as a substitute. If neither the demonstrated object nor a same-class object was available, the robots were expected to recognize that the task was infeasible and remain inactive. The items used in the experiment and their corresponding classes are shown in Fig. S3.

As shown in Table 1, in the 11 scenarios where the exact demonstrated item was available, the robots successfully delivered the exact item an average of 9.33 times. This result highlighted the system's precision in replicating demonstrated actions under ideal conditions. In the 13 scenarios where the exact item was unavailable, but an alternative item within the same class was present, the system correctly identified and delivered the substitute item an average of 11.33 times. This demonstrated the framework's adaptability in selecting appropriate alternatives, effectively handling less-than-ideal conditions whereas still achieving task goals. Finally, in the six scenarios where no relevant items were available, the robots correctly recognized the task's infeasibility and remained inactive an average of 5.67 times. This underscored the system's ability to skip actions when necessary, preventing errors and avoiding risky operations.

Fig. 4D illustrates the team's imitative behaviors that best adapted to these three conditions. First, the learner team successfully selected and delivered the yellow tape as demonstrated, despite the presence of another item in the same class (black tape). In this scenario, the proposed IAIL framework allocated tasks to learner robots with functionalities most similar to those of the demonstrator team: Cuboat versus Tello, Single-Arm versus Dual-Arm, and Diablo versus Spark. This indicated that the framework effectively enables team-to-team imitation. When the yellow tape from the demonstration was removed, the black tape was identified as a substitute. However, this item was out of reach for the Single-Arm robot. As a result, Pepper, which possesses both manipulation

and navigation capabilities, was assigned to fetch and deliver the item (see Fig. 4D(i)). In this case, the IAIL framework not only adapted to changes in environmental conditions but also ensured efficient task allocation. When the black tape was also removed (Fig. 4D(ii)), no relevant items were available. The imitation was considered successful because the robots remained inactive, as performing any action in this scenario would lead to incorrect or unintended outcomes. The IAIL framework also demonstrated robustness to variations in the composition of the learner team. For example, when Cuboat was absent (Fig. 4D(iii)), Diablo substituted its role for monitoring, leaving Pepper to receive and deliver the tape. This emergent task allocation highlighted the strong generalizability of the proposed IAIL framework and its ability to bridge the skill sets of heterogeneous robots using shared intentions. The videos of the IL results shown in Fig. 4D(i), D(ii), and D(iii) are provided in the Supplementary Materials Movie S1.

Flexibility in Robot Role Assignments

To further assess the flexibility and adaptability of our framework, we analyzed how tasks were distributed across different robots in the learner team. Table 2 presents the proportion of scenarios in which each robot was assigned to a given task phase (user monitoring, item picking, and item delivery), as well as the percentage of task assignments that fell within each robot's functional capabilities, confirming the validity of the assigned roles.

Table 2: **Distribution of task assignments across the learner robot team.** Each cell indicates the proportion of scenarios in which the robot was assigned the specified task. The last column shows the percentage of assigned tasks that were within the robot's capability scope. Percentages are computed from evaluation scenarios in which the task was completed successfully.

Robot	Monitor User Activity	Pick Item	Deliver Item	Rate of Assigning Feasible Tasks
Cuboat	38%	–	–	100%
Diablo	62%	–	29%	100%
Single-Arm	–	50%	–	100%
Pepper	–	50%	50% + 21%[†]	100%

[†] For Pepper, 50% of deliveries were of items it picked itself; 21% were handovers from Single-Arm, used when Diablo was unavailable for delivery.

This analysis provided two key insights. First, across all scenarios, assigned tasks were strictly within each robot's physical and functional capabilities, resulting in a 100% feasibility rate. For example, only mobile robots were tasked with navigation and delivery, whereas fixed-base manipulators were assigned only object-picking roles. This confirmed that the system respected embodiment constraints and avoided unsafe or infeasible actions. Second, the distribution of roles varied with

environmental configuration and team composition. For example, Cuboat and Diablo were both assigned to monitoring in different scenarios (38% and 62%, respectively), and item delivery was handled by Diablo (29%) and Pepper (71%). In 21% of scenarios, Pepper delivered items handed over by Single-Arm, typically when Diablo was unavailable, indicating that the system adapted task assignments in response to agent availability and context. Although the assignments remained feasible, the resulting role allocation was not hard-coded and instead reflected context-sensitive decision making.

Together, these results indicated that the IAIL framework enabled flexible and adaptive role assignments that respected robot capabilities and responded to team composition. This context-aware allocation strategy demonstrated the system's ability to generalize across embodiments and conditions, which was essential for scalable deployment in dynamic, heterogeneous robot systems.

Quantitative Evaluation of the Latent Intention Space

To further assess the internal structure and robustness of the shared intention space learned by IAIL, we conducted a quantitative analysis of the latent embedding distributions across tasks and robot embodiments. We measured semantic separation between task types using intra-class and inter-class cosine distances in the latent space. We also assessed embodiment invariance by computing cross-embodiment alignment errors.

For this analysis, we randomly sampled three unseen trajectories per robot per task, yielding a total of 120 test samples. These samples were not used during training and spanned the same task set evaluated in the real-world experiments. To provide an intuitive understanding of the latent intention space, we applied t-distributed Stochastic Neighbor Embedding (t-SNE) to project the high-dimensional intention embeddings into a two-dimensional space. The resulting visualization is shown in fig. S5 in the Supplementary Materials.

Semantic separation across tasks

We computed pairwise cosine distances among latent embeddings corresponding to five high-level task phases: Monitoring, Picking, Prepare_handover, Complete_handover, and Delivery. Intra-class distance was defined as the average cosine distance between embeddings of the same task type, whereas inter-class distance was computed across different task types. To summarize overall task separability, we report the semantic separation ratio: the mean inter-class distance divided by the mean intra-class distance, a form commonly used in unsupervised clustering evaluation (*42, 43*).

For the Picking phase, we also computed intra-class distances at two levels of semantic granularity by treating each distinct object as a separate task ("same item") and by grouping objects into broader semantic categories ("same-class item"), such as grouping all cup variants into a single category. The item instances and classes are shown in Fig. S3. The latter reflected the grouping

Table 3: **Quantitative analysis of latent intention space structure.** Each row reports the mean ± standard deviation of cosine distances for intra-class clustering and cross-embodiment alignment across tasks. The standard deviation is computed using models trained with three different random seeds. Lower values indicate tighter clustering and stronger embodiment invariance. The bottom rows report global inter-class distance and the resulting semantic separation and embodiment alignment ratios, which reflect the overall organization and generalizability of the latent space.

Task type	**Intra-class distance**		**Cross-embodiment error**	
Monitoring				
M_1	0.375 ± 0.006		0.499 ± 0.008	
M_2	0.356 ± 0.002		0.475 ± 0.002	
M_3	0.290 ± 0.007		0.484 ± 0.012	
M_4	0.276 ± 0.003		0.460 ± 0.005	
Picking	*same item*	*same-class item*	*same item*	*same-class item*
wood block	0.018 ± 0.003	0.230 ± 0.014	0.055 ± 0.011	0.162 ± 0.021
cup	0.110 ± 0.044	0.499 ± 0.034	0.297 ± 0.030	0.423 ± 0.010
glue gun	0.022 ± 0.01	0.280 ± 0.005	0.051 ± 0.005	0.373 ± 0.007
tape	0.019 ± 0.003	0.235 ± 0.027	0.061 ± 0.009	0.358 ± 0.005
paint	0.120 ± 0.128	0.271 ± 0.006	0.103 ± 0.076	0.180 ± 0.053
Prepare_handover	0.352 ± 0.007		0.469 ± 0.009	
Complete_handover	0.226 ± 0.006		0.301 ± 0.008	
Delivery				
D_1	0.023 ± 0.003		0.031 ± 0.004	
D_2	0.023 ± 0.002		0.030 ± 0.002	
Global inter-class distance	**0.997 ± 0.003**			
Semantic separation ratio	**3.764**			
Embodiment alignment ratio	**3.046**			

used for success rate computation in Table 1 and was used for calculating the overall separation ratio.

As shown in Table 3, the global inter-class distance was high (0.997 ± 0.003), indicating that task-specific latent representations were well-separated and nearly orthogonal. In contrast, intra-class distances were substantially lower across all task phases. For instance, the Monitoring tasks exhibited intra-class distances ranging from 0.276 to 0.375, whereas Picking tasks ranged from 0.23 to 0.499, reflecting variability due to differences in object texture and geometry in each class. The Handover phases showed distances from 0.226 to 0.352, and Delivery tasks demonstrated the tightest clustering, with distances 0.023. These values collectively yielded an average separation ratio of

3.764, confirming that IAIL's intention space formed compact, task-specific clusters. Even for picking cups, which involved four visually dissimilar cup types, the embeddings formed relatively tight latent clusters (up to 0.499 ± 0.034), indicating that high-level intent is preserved despite appearance variation. This well-structured latent space directly supported IAIL's ability to reliably associate intentions and retrieve appropriate policies, as reflected in its overall task success rate of 92% in Table 1.

Fine-grained object structure

When comparing intra-class distances between same-item and same-class-item groupings in the Picking phase, we observed that same-item distances were consistently much lower. For example, the intra-class distance for picking a specific cup was 0.11 ± 0.044, compared to 0.499 ± 0.034 when considering the class to consist of of four types of cups. This suggested that the latent space preserved fine-grained semantic distinctions and formed tighter clusters around specific object instances.

This structure was functionally important, as it enabled IAIL to preferentially select same-item demonstrations (rather than same-class ones) whenever they were available during latent-similarity-based motion association.. As shown in Table 1, this preference translated into high adaptation accuracy, achieving 85% success when the same item was available, and 87% when only same-class items were accessible. These results confirmed that the intention space encoded discriminative features at both coarse (task-type) and fine (object-instance) levels, which was essential for robust and flexible motion imitation.

Cross-embodiment alignment

To evaluate the embodiment invariance of the latent intention space, we defined the cross-embodiment alignment error, in analogy to the intra-class distance used in classic cluster validation metrics. Specifically, we computed the mean cosine distance among the embedding centroids of different robots performing the same task. This metric quantified how tightly intention representations from heterogeneous embodiments clustered within each task category, with lower values indicating stronger alignment across robots.

As shown in Table 3, monitoring tasks exhibited alignment errors ranging from 0.460 to 0.499, approximately half the inter-class distance, indicating consistent intention encoding despite substantial differences in robot morphology. The Picking phase also demonstrated robust cross-embodiment consistency (errors from 0.162 to 0.423), even across objects with distinct shapes and grasp strategies. The Delivery phase achieved the lowest alignment errors (0.030–0.031), indicating nearly identical latent encoding across robots. To summarize embodiment invariance across all tasks, we computed the embodiment alignment ratio, defined analogously to the separation ratio as

the mean inter-class distance divided by the mean cross-embodiment alignment error. The resulting ratio of 3.046 confirmed that robots with widely varying embodiments consistently encoded shared task intentions in a unified latent space.

Furthermore, consistent with the trend observed in semantic separation, we found that cross-embodiment alignment errors were smaller when comparing robots picking the same specific item than when aggregating items within the same semantic class. This further reinforced the latent space's capacity to preserve object-level specificity across embodiments.

This cross-embodiment alignment was functionally critical for enabling IAIL's flexible role allocation capabilities, as demonstrated in Table 2. In scenarios where a robot became unavailable or suboptimal, other robots can step in by evaluating the similarity of their candidate motions in the latent intention space. This shared representation enabled dynamic task redistribution based on individual capabilities and availability, with minimal coordination overhead.

Comparisons with baseline methods

Recent advances in cross-embodiment transfer have explored adaptation between individual robots (*7, 18, 44, 45*). However, these approaches predominantly assume a fixed one-to-one correspondence between a demonstrator and a learner, and they do not directly address capability-aware role assignment and validity guarantees required for imitation in heterogeneous robot teams. To contextualize our contributions within existing frameworks, we evaluated our method in a simplified one-to-one agent setting, where the goal was to generate valid motions that achieved the same intended outcome as the demonstration. We benchmarked our approach against two representative paradigms in agent-to-agent imitation: density-based mapping and description-based translation.

Baseline methods

We compared IAIL against two representative baselines, a density-based approach adapted from (*18*) and a description-based approach that uses natural language as an intermediate representation (*46, 47, 48, 49, 50, 51, 52*).

For the density-based approach, the baseline learned unsupervised correspondences between robot behaviors by aligning the distributional divergence of skills extracted from motor trajectories, where skills are defined as temporally extended behaviors. The underlying assumption was that different robots share similar skill structures for accomplishing comparable tasks, even in the presence of substantial morphological differences. To align skill distributions across robots, the method employed a cycle-consistency loss that maximizes the likelihood of both forward and reverse translations between the source and target domains. During inference, the learner robot encoded the demonstrator's trajectory into a latent skill using the demonstrator's encoder, and then reconstructed an executable trajectory using its own decoder. In our implementation, we

adopted the same encoder and decoder architectures as those used in the IAIL framework to ensure comparability. This baseline operated entirely in an unsupervised manner and does not rely on task labels or annotations. It aimed to establish correspondences purely based on structural similarities in skill execution. In our experiments, we evaluated this method using robot pairs with differing task distributions and capabilities. Although the method could mitigate embodiment mismatches to some extent, its performance degraded substantially when the demonstrator and learner differed in task distributions, due to its lack of semantic grounding in task intentions.

For the description-based approach, the baseline represented a class of methods that leverage language-conditioned policy learning, in which natural language serves as an intermediate representation for imitation (*46, 47, 48, 49, 50, 51, 52*). Rather than establishing direct mappings between latent action spaces or trajectories, this paradigm enabled communication between the demonstrator and learner robot through language. This design facilitated zero-shot behavior transfer across embodiments by grounding actions in a shared semantic representation. In this baseline, demonstrated motions were first encoded into textual descriptions that summarize the observed behavior. These descriptions are then transmitted to the learner robot, which decoded them into executable motor actions intended to fulfill the described objective. To train the encoder, we followed a contrastive learning framework introduced in (*53*) to align the motion and the annotation. The decoder was implemented as a language-conditioned policy trained to reproduce the paired motor trajectory given the input annotation, independently for each learner robot. We use the same annotated dataset to train both the encoder and decoder as in the IAIL framework, ensuring that the level of supervision is matched for a fair comparison. This baseline could be viewed as a simplified variant of IAIL that omits the intention association mechanism and therefore does not account for the learner robot's capabilities during action selection. In our experiments, we evaluated this method using robot pairs with differing task distributions and embodiment constraints. Unlike the density-based baseline, the description-based method was less sensitive to discrepancies in task distributions, as its semantic grounding provides a degree of robustness. However, it lacks an explicit mechanism to assess whether the learner robot can interpret the annotations correctly or execute the decoded actions feasibly. As a result, when the physical capabilities of the learner do not align with those assumed in the demonstration, the method frequently produced invalid or ineffective behaviors. This issue is particularly pronounced in heterogeneous settings, where we observed substantial performance variability across robot pairs with different capabilities.

Simulation tasks

The evaluation was conducted on two simulated tasks analogous to the user monitoring and item picking phases. Simulation-based evaluation was less affected by real-world disturbances, such as limited training data, sensor noise, or motor inaccuracies, ensuring that the results accurately reflected the core capabilities of each method. In the monitoring task, we focused on the effects

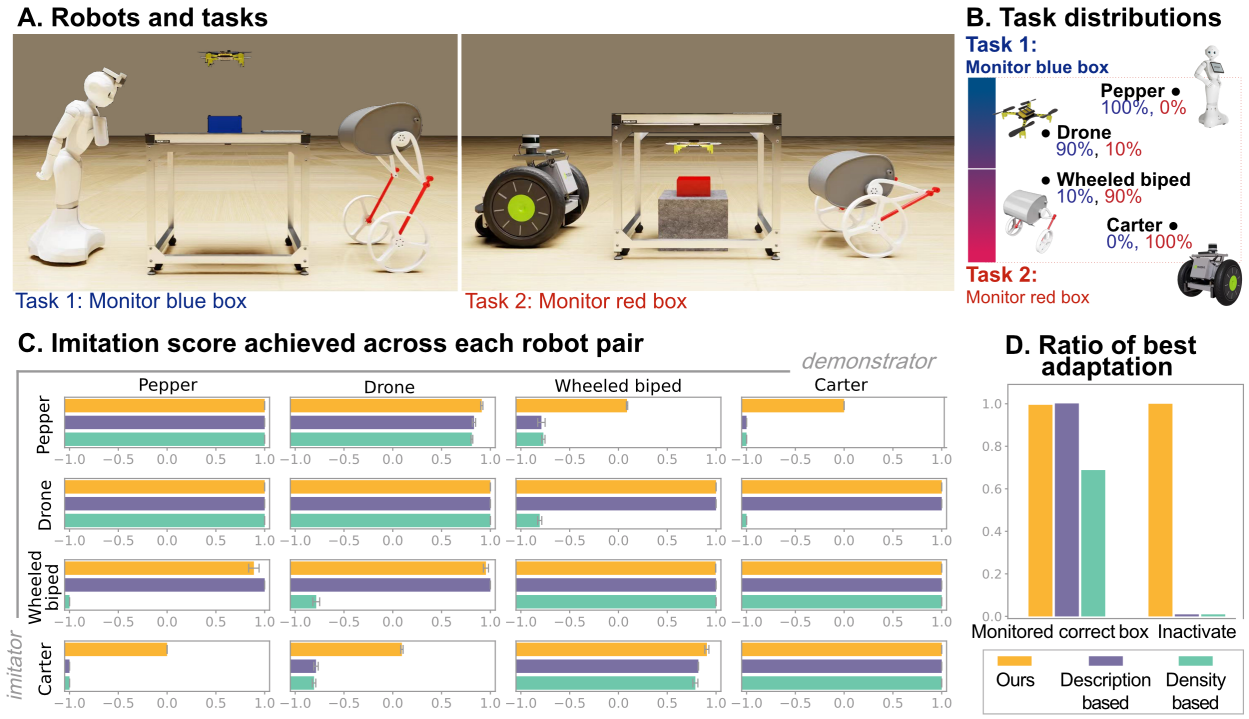

Figure 5: **The simulation study involves a monitoring task designed to evaluate imitation performance across robot pairs with different task distributions in their datasets.** **(A)** Example poses of the robots monitoring two boxes: a blue box on the table and a red box under the table. **(B)** Trajectory distributions for monitoring different boxes in each robot's dataset: Due to embodiment limitations, Pepper can only see the blue box, whereas Carter can only see the red one. Drone and Wheeled Biped can see both, but exhibit different preferences. **(C)** The average task score achieved by each demonstrator–learner robot pair. Statistics: Data are presented as mean ± standard deviation (SD). Each bar represents the mean score across N = 1,500 evaluation runs. Detailed per-pair statistics including effect sizes and confidence intervals are provided in supplementary materials. **(D)** the ratio of selecting the most appropriate action. The evaluation was performed three times using models trained with three different random seeds.

of robot variations, whereas in the item picking task, we focused on the effects of environmental variations. The detailed simulation experiment setup is provided in the Supplementary Materials.

The setup for the simulated target monitoring task is illustrated in Fig. 5A. We used four robots (Pepper, Drone, Carter, and Wheeled Biped) to monitor two targets: a blue box on a table and a red box under a table. Owing to their embodiment constraints, each robot had a different preference for monitoring these targets, influencing their action distribution during the training and evaluation phases (Fig. 5B). For example, 100% of the actions performed by Pepper involved monitoring the blue box, whereas 100% of the actions performed by Carter involved monitoring the red box. For the Drone, 90% and 10% of its actions involved the blue box and the red box, respectively,

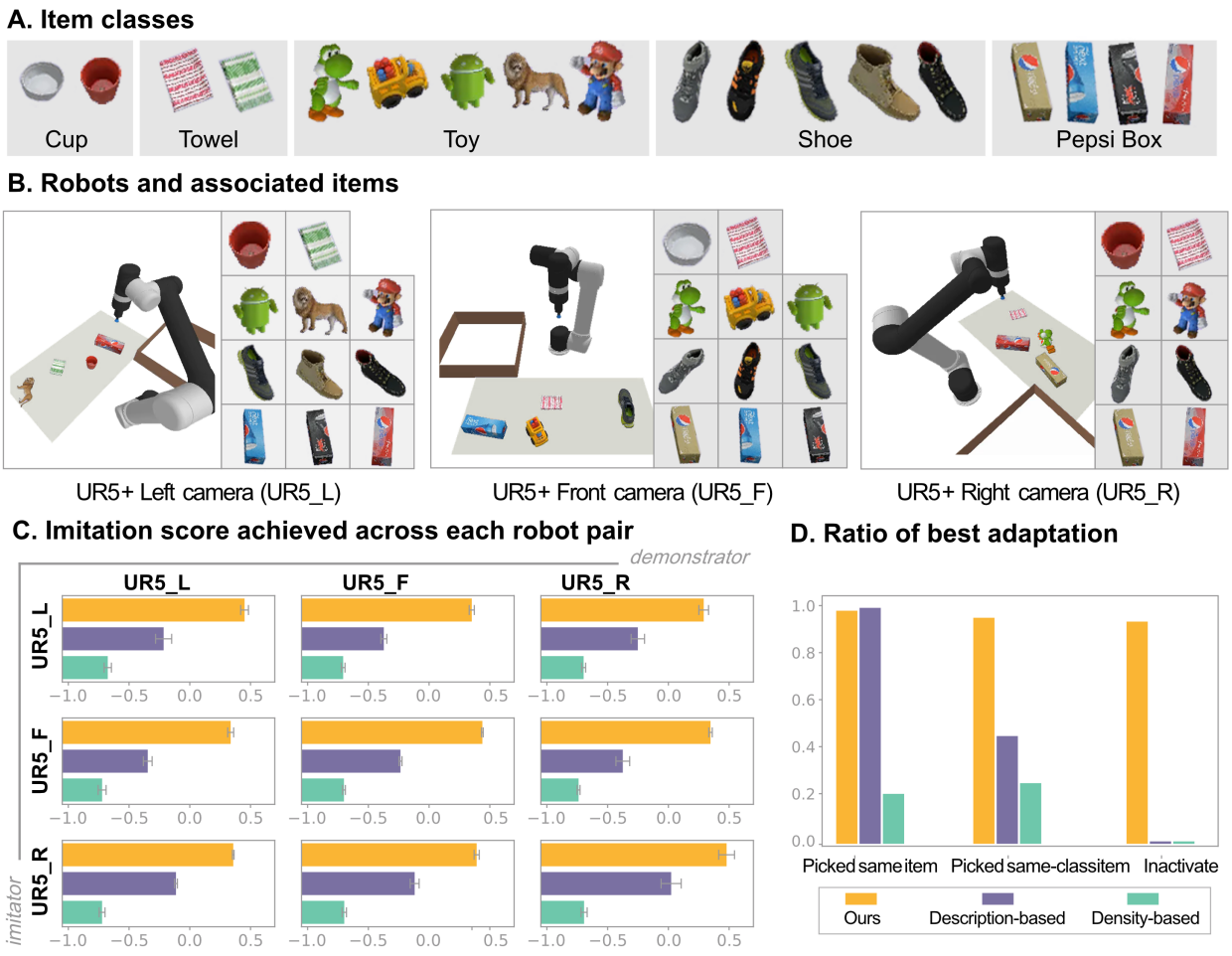

Figure 6: **The simulation study involving selecting a proper item based on demonstrations under different environmental conditions.** **(A)** The 18 items used in the task, categorized into five classes. **(B)**Variations in the task environment include different camera angles and selectable item types. An example image from each camera is shown on the left, and the selectable item list associated with each robot is displayed on the right. In each trial, four items are randomly chosen from the associated item list and placed on the table. **(C)** The average task score achieved by each demonstrator-learner robot pair. Data are mean ± SD. N = 1,500. Detailed per-pair statistics including effect sizes and confidence intervals are provided in supplementary materials. and **(D)** the ratio of selecting the most appropriate item. The evaluation was performed three times using models trained with three different random seeds.

whereas for the Wheeled Biped, 90% and 10% of its actions involved the red box and the blue box, respectively.

The setup for the simulated item picking task is illustrated in Fig. 6B. In this task, we used three UR5 (*54*) robot arms from Universal Robots. The robots had identical kinematic structures but different camera viewpoints. Each robot was assigned a specific set of items and was tasked

with picking items from the table in front of it. At each iteration, four items were randomly selected from the assigned set. They were placed on the table with random positions and orientations. The full collection comprised 18 items grouped into five classes, as shown in Fig. 6A.

Performance comparison

We evaluated the imitation performance between all possible demonstrator-learner robot pairs. Based on the alignment results after execution, we assigned a score of 1 if the learner robot monitored the correct target or picked the same item as in the demonstration, a score of 0.5 if the learner robot picked an item within the same class, a score of 0 for skipping the action, and a score of -1 for performing an irrelevant action. Ideally, the robot should have performed the demonstrated task when possible, chosen an alternative if an exact match was unfeasible, and remain inactive if the task cannot be completed.

The average scores achieved by each demonstrator-learner pair in the monitoring task and item picking task are presented in Fig. 5C and Fig. 6C, respectively. Each pair was evaluated over 500 runs, and the evaluation was repeated three times using models trained with different random seeds.

The results of the monitoring task are shown in Fig. 5C. The density-based method was highly sensitive to differences in action distributions between robots. It performed well when the action distributions of the demonstrator and learner are similar, as observed in the Pepper–Drone and Carter–Wheeled Biped pairs. However, its performance degraded substantially when the task distributions in the training data diverge (Fig. 5B). These challenging cases include Pepper–Wheeled Biped, Pepper–Carter, Drone–Wheeled Biped, and Drone–Carter pairs, with each robot acting as both learner and demonstrator, for a total of eight imitation cases. To validate this observation, we conducted a two-sided Welch's t-tests for these eight robot pairs. For each pair, the test was performed on the $1,500$ scores (500 runs, repeated three times using models trained with different random seeds) achieved by IAIL and the density-based method. Across these eight cases with substantial distributional mismatch, IAIL significantly outperformed the density-based baseline (all $p < 0.001$). The unweighted average score difference across the eight cases was $\bar{\Delta} = 1.40$ with 95% CI $[1.01, 1.79]$ (SD = 0.47, range $[0.86, 2.00]$). For transparency, per-case statistics, including Δ with 95% CI, t, df, p, and Cohen's d (unequal-variance), are reported in table S3.

In contrast, the description-based method demonstrated robustness to distribution shifts but struggled when the learner robot lacks the capabilities required to replicate the demonstrator's actions. For example, it succeeded when Carter serves as the demonstrator and Drone as the learner, but failed in the reverse scenario due to Carter's limited ability to monitor the blue box from an aerial viewpoint. Similar capability mismatches occurred in four imitation cases: Pepper–Wheeled Biped, Pepper–Carter, Carter–Pepper, and Carter–Drone, where the former (as learner) lacks the capability to complete some tasks demonstrated by the latter (as demonstrator). In these four cases, Welch's t-tests again showed that IAIL significantly outperformed the description-based baseline

(all $p < 0.001$). The unweighted average score difference of the four cases was $\bar{\Delta} = 0.94$ with 95% CI $[0.84, 1.04]$ (SD = 0.063, range $[0.85, 1.00]$). Full per-case statistics (Δ with 95% CI, t, df, p, and Cohen's d) are reported in table S4.

Both the density-based and description-based methods share a key limitation: they cannot detect when a demonstrated task is impossible for the learner robot to complete. This issue is particularly evident with the Pepper–Carter pair, for which the average score was -1, indicating consistent execution of incorrect actions by the robots, which could lead to failure or unexpected errors in real-world scenarios. These findings were supported by the ratio of the best adaptation in different scenarios, as shown in Fig. 5D. With the description-based method, the demonstrated task was correctly executed when the learner robot can perform the task; however, this method failed to detect infeasible tasks. The performance of the density-based method was even worse, as it was also affected by the action distributions of the robots.

The results of the item picking task are shown in Fig. 6C. There was a noticeable decline in the scores of each robot pair in the item picking task compared with the scores in the monitoring task. This decline was due to increased task complexity, higher-dimensional state and action spaces.

The density-based method yielded the lowest scores across all robot pairs. The description-based method performed better but still fails to achieve an average score of 0, indicating that irrelevant or incorrect actions were frequently executed. To validate these observations, we performed Welch's t-tests across all nine imitation cases involving different robot pairs in the item-picking task. The performance differences between IAIL and both baseline methods were statistically significant (all $p < 0.001$). For the density-based method, the unweighted average score difference across the nine cases was $\bar{\Delta} = 1.11$ with 95% CI $[1.08, 1.14]$ (SD = 0.04, range $[1.02, 1.18]$). For the description-based method, the unweighted average score difference was $\bar{\Delta} = 0.63$ with 95% CI $[0.55, 0.70]$ (SD = 0.10, range $[0.47, 0.74]$). Full per-case statistics (Δ with 95% CI, t, df, p, and Cohen's d) are reported in table S5 and S6.

A similar pattern was observed in the ratio of the best actions performed, as shown in Fig. 6D, which reflected the trend in Fig. 6C. The description-based method achieved good results when the exact demonstrated item is available. However, its performance decreased substantially when adapting to an alternative item within the same class. The density-based method struggled in both scenarios. Additionally, both methods are unable to handle cases in which the demonstrated actions were impossible to perform.

For both tasks, our method consistently achieved the highest scores for almost all robot pairs and across all types of imitation scenarios. Unlike the density-based method, which relies solely on aligning the action distributions of the learner and demonstrator robots, our approach leveraged the semantic information embedded in the robot actions. This enabled more accurate and meaningful action association. Moreover, in contrast to the description-based method, which relies on descriptions without considering the learner's capabilities, our method integrated context-aware

motion generation and intention-based association. This ensured that the selected actions were both executable for the learner robot and aligned with the demonstration, enabling greater adaptability to changing conditions.

In addition to the performance issues, the baseline methods had limited scalability for supporting IL between robot teams. Specifically, they lack the ability to compare motions across different robots and, therefore, cannot effectively address the role-assignment problem in team-to-team imitation scenarios. This limitation notably restricted their applicability in complex, multi-robot environments. In contrast, our IAIL framework overcomes these challenges. By leveraging semantic motion understanding and association in the intention space, IAIL can seamlessly handle IL between heterogeneous teams of robots, regardless of their numbers, types, or configurations. It dynamically assigns roles within the learner team based on individual capabilities and adapts to evolving task requirements without relying on predefined distributions or rigid descriptions. This flexibility enables IAIL to scale effectively across diverse robotic systems, ensuring robust and adaptable performance in real-world scenarios.

DISCUSSION

In this work, we introduced the IAIL framework, which enhances the adaptability, generalizability, and scalability of IL for heterogeneous robotic systems, thereby extending its applicability to dynamic and complex real-world scenarios. The framework leveraged the intentions underlying robot motions to enable heterogeneous robots to automatically adjust behaviors based on demonstrator-learner relations, accommodating variations in tasks, robot types, and operating environments. Our results demonstrated that the IAIL framework substantially improves robots' ability to learn and perform tasks by imitating the behaviors of other robots. It also reduces reliance on task-specific demonstrations requiring precise alignment with target configurations.

This adaptability is especially critical for heterogeneous robot teams, where robots with diverse characteristics must collaborate to complete a wide range of tasks under changing conditions. In addition to improving task performance, the framework's scalability allows it to support diverse robot team configurations, making it particularly well-suited for real-world applications.

Recent advances have emphasized learning from large-scale, heterogeneous datasets collected from diverse robot platforms performing a wide range of tasks in varied environments, such as Open X-Embodiment (*50*), Octo (*51*), OpenVLA (*52*), and HPT (*55*). These efforts aim to develop general-purpose representations that can accelerate task learning for individual robots and enable implicit knowledge sharing across contexts. However, these methods primarily focus on learning universal policies or representations, without explicitly tackling the challenge of transferring task-solving behaviors between robots with different embodiments, an ability that is essential for effective cross-robot generalization. Inspired by human imitation mechanisms, the IAIL framework bridges

this gap by using high-level motion intentions to enable direct behavior association and adaptation in a shared representation space (*56*). This formulation preserves task-agnostic representations and enabling explicit and flexible skill transfer across heterogeneous robot embodiments. By allowing robots to interpret, imitate, and adapt the task-solving strategies of others, IAIL fosters more efficient collaboration and broader generalization in diverse, real-world robotic systems.

Although our framework leverages language to explicitly represent intention, we acknowledge that intentions can also be modeled implicitly through goal-inference processes, in which observers deduce goals from observational or behavioral cues (*57, 58, 59*). However, in the context of heterogeneous robot teams, relying solely on implicit behavioral cues is challenging, as the observable forms of motion and action modalities differ notably between agents. In this setting, our linguistic approach offers a complementary and robust perspective, providing a shared, high-level code for intention that bridges embodiment gaps where direct visual or motion correspondence is unreliable. Furthermore, by grounding imitation in linguistically specified intent, our framework is closely related to the objective of robot motion legibility in the robotics literature, which seeks to make a robot's goals efficiently inferable from its behavior (*60, 61*). By organizing robot behaviors in a shared intention space aligned with human-understandable descriptors, IAIL supports both legibility and predictability, which is particularly advantageous in collaborative scenarios, especially when humans are involved (*62, 63*).

As a promising extension, integrating large language models (LLMs) could further unlock the potential of the IAIL framework in robotic skill acquisition and task allocation. By utilizing the annotation encoder trained alongside the motion encoder, the IAIL framework can process language instructions as seamlessly as it processes demonstration trajectories. This capability enables smooth integration with LLMs, expanding the framework's ability to interpret and act upon language-based inputs. When given a language instruction, the annotation encoder f_ξ extracts the underlying intention and identifies the motion most closely aligned in the intention space. This allows the framework to select the action most likely to achieve the desired outcome based on the inferred intention. If collecting trajectory demonstrations becomes impractical, LLMs can be leveraged to automatically generate instructions as demonstrations. The only required adjustment is switching the encoder from f_ψ to f_ξ in the computation of V_{task}. As a preliminary exploration, fig. S5 shows an example of the integrated pipeline for task planning and execution with a learner robot team, and a video of this example is provided in the Supplementary Materials Movie S3.

One limitation of our framework is identifying the decision boundary to determine when the generated actions do not satisfy the demonstrated task, requiring the robot to remain inactive. Currently, we apply a fixed threshold for all the robots within the group. Although this approach makes the system more generalizable and user-friendly, it may hinder performance since the fixed threshold might not be optimal for every robot. This process could be further enhanced by automatically assigning and tuning parameters for each robot.

In the future, we intend to explore more complex imitation scenarios in which demonstrations may need to be broken down into more manageable components or reassembled according to the capabilities of the learner robots. Furthermore, we aim to incorporate multimodal inputs, combining robot trajectory data, text descriptions, and sensor feedback to enable a more comprehensive understanding of the task, thereby enhancing the flexibility and applicability of the framework. Future efforts will focus on refining the framework to improve its user-friendliness and accessibility. For instance, we plan to devise methods for automatically annotating robot trajectories and tuning the hyperparameters of our model. These methods would simplify the creation of training data and reduce technical barriers, making the methodology available to a broader audience of researchers and practitioners.

MATERIALS AND METHODS

As described in the Results section, our IAIL framework operates through three key processes: context-aware motion generation, motion intention extraction, and motion association. These processes involve training three types of models: the motion generator (p_{θ_i}), the motion encoder (p_{ψ_i}), and the annotation encoder (f_ξ). The following subsections provide detailed explanations of the training procedures for these models, the method for associating actions between robots, and the approach for enabling imitation within robot teams.

Learning process of the motion generator

The motion generator p_{θ_i} is responsible for assessing each robot's capabilities in various contexts. This capability is manifested through the action trajectories that p_{θ_i} can generate for each robot state. We train the motion generator with a precollected expert dataset that contains action trajectories completing diverse tasks within the robot's domain. The objective of the motion generator is to reproduce the action trajectories from the dataset. As a result, we can generate safe and executable motion trajectories that enable diverse goals to be achieved considering the context of each robot.

Collecting the Robot Trajectory Datasets

The dataset for each robot is collected independently by human experts. The experts first define a set of tasks and then perform these tasks using the robot. We repeat each task multiple times, collecting trajectories under different conditions, such as varying object types, locations, or orientations, and different robot initial positions. Whenever possible, we employ different trajectories to complete the same task, ensuring a comprehensive coverage of the variations in the task environment. We record the robot trajectories $\tau = s, \mathbf{a}$, which consists of the initial state s and the sequence of

actions $\mathbf{a} = (a_0, \ldots, a_H)$ executed by the robot. We denote the dataset for each robot as D_i, and the combined dataset from all robots as $\mathcal{D} = \bigcup_{i=1}^{N} D_i$.

Training the Generator

The motion generator for robot i is modeled as a latent-variable, state-conditioned generative model $p_{\theta_i}(\mathbf{a}|s, z)$, where s is the robot's state and z is a latent variable capturing the underlying variations in the action sequences. This generative model is trained to reproduce action sequences $\mathbf{a}$ in the precollected dataset given different states s. In this work, we employ a variational autoencoder (VAE) (*64*) to train the generative model. Alternative methods such as generative adversarial networks (GANs) (*65*) and diffusion models (*66*) could also be utilized.

The objective of the VAE for robot i is as follows:

$$\arg\max_{\vartheta_i, \theta_i} \mathop{\mathbb{E}}_{(s,\mathbf{a})\sim D_i} \Big[\mathbb{E}_{q_{\vartheta_i}(z|s,\mathbf{a})} [\log(p_{\theta_i}(\mathbf{a}|s, z)) - \beta_{KL}(q_{\vartheta_i}(z|s, \mathbf{a})||p(z))] \Big], \tag{1}$$

where $q_{\vartheta_i}(z|s, \mathbf{a})$ denotes the variational posterior distribution of the encoder. $p_{\theta_i}(\mathbf{a}|s, z)$ represents the decoder's posterior distribution, which serves as our generative model. $p(z)$ is the prior distribution over the latent variable, which is typically modeled as a standard normal distribution, and β_{KL} is a hyperparameter that balances the reconstruction loss and the Kullback–Leibler (KL) divergence between the posterior and prior distributions. The generative model for each robot is trained independently with its corresponding dataset D_i. Please refer to (*64*) for a more detailed description of the VAE.

In theory, the generative model should produce action sequences that lie within the distribution of the training dataset. This property allows for the sampling of safe and executable actions for performing diverse tasks within each robot's domain. However, in practice, the model may generate out-of-distribution (OOD) actions due to interpolation between data samples (*67, 64*). These OOD actions can lead the robot to unexpected states, introducing uncertainties during task execution. To mitigate this issue, it is essential to identify and eliminate OOD actions from the generated actions. Such actions are identified and eliminated through the motion encoding and associating steps, which are detailed in the next section.

Joint Learning of the Motion and Annotation Encoders

The motion encoder p_{θ_i} for robot i is responsible for extracting the intentions underlying the robot's motions. These motion intentions are defined by human-annotated language descriptions that capture the core objectives of the motions. The motion intentions serve as unified motion representations across all robots, enabling each robot to interpret and compare its own motions with those of other robots. The motion encoder achieves this goal by mapping robot motions to a

latent space, also referred to as the intention space, where each dimension corresponds to a specific motion intention.

We employ a shared annotation encoder $f_\xi(l)$ to process the language annotations. The robot-specific motion encoder p_{θ_i} and the shared annotation encoder $f_\xi(l)$ are trained jointly using a contrastive learning approach. The aim of this joint training is to align the motion encodings with their corresponding annotation encodings. By doing so, we ensure that motions with similar intentions are encoded closely together within the intention space, even if they originate from different robots.

Annotating the Robot Trajectory Datasets

We annotate the trajectories in the precollected datasets to construct the motion-annotation pairs required to train the motion encoders. Each trajectory is assigned 3–5 language descriptions with varying levels of abstraction, ranging from detailed to concise. For instance, consider a trajectory where a robotic arm picks up a white paper cup. The annotations for this trajectory include "pick up a white paper cup", "pick up a paper cup", "pick up a white cup" and "pick up a cup". To further enrich the annotations, we employ different synonyms and phrasings to augment the descriptions, ensuring that the key attributes of each trajectory are captured in diverse ways.

The OOD actions generated by the motion generator p_{θ_i} are not included in the initial dataset. Consequently, their encodings in the intention space do not accurately reflect the true outcomes of the motion executions. To correctly interpret the OOD actions, we extend the dataset by incorporating actions sampled directly from the motion generator $p_{\theta_i}(\mathbf{a}|s, z)$ using random z values drawn from the prior distribution $p(z)$. We annotate these sampled actions as "unknown" because their precise outcomes are uncertain.

In the annotated and extended dataset $\mathcal{D}^l$, OOD actions are annotated solely as "unknown," whereas in-distribution data include both specific annotations and "unknown" actions. By training the encoders with this extended dataset, OOD actions are encoded much closer to the "unknown" label in the intention space, enabling effective identification of these actions.

Joint Training of the Motion and Annotation Encoders

The motion encoder $f_{\psi_i}(\tau)$ processes and maps the trajectory of robot i to a multidimensional encoding. The annotation encoder $f_\xi(l)$ maps language annotations to encodings of the same dimensionality as those produced by the motion encoder. We maximize the mutual information between these encodings via a contrastive learning approach (*53*). In this approach, the encoding distance between correct (positive) motion-annotation pairs is minimized, whereas the encoding distance between incorrect (negative) motion-annotation pairs is maximized. The loss function is

defined as follows:

$$\mathcal{L}_{\psi,\xi} = \mathcal{L}^{\tau\to l}_{\psi,\xi} + \mathcal{L}^{l\to\tau}_{\psi,\xi}, \tag{2}$$

where

$$\mathcal{L}^{\tau\to l}_{\psi,\xi} = \underset{\tau_i,l_i\sim\mathcal{D}^l}{-\mathbb{E}}\left[log\frac{\mathcal{E}(\tau_i,l_i)}{\mathcal{E}(\tau_i,l_i)+\sum_{l_j\sim\mathcal{D}^l}^{N_b-1}\mathcal{E}(\tau_i,l_j)}\right], \tag{3}$$

and

$$\mathcal{L}^{l\to\tau}_{\psi,\xi} = \underset{\tau_i,l_i\sim\mathcal{D}^l}{-\mathbb{E}}\left[log\frac{\mathcal{E}(\tau_i,l_i)}{\mathcal{E}(\tau_i,l_i)+\sum_{\tau_j\sim\mathcal{D}^l}^{N_b-1}\mathcal{E}(\tau_j,l_i)}\right]. \tag{4}$$

Here, $\mathcal{D}^l$ represents the union of the annotated datasets of all the robots, $\tau_i = (s, \mathbf{a})$ is a sampled trajectory, and l_i is the annotation of τ_i. We construct negative motion-annotation pairs by randomly sampling $N_b - 1$ elements from $\mathcal{D}^l$ (sampling l_j for $\mathcal{L}^{\tau\to l}_{\psi,\xi}$ and τ_j for $\mathcal{L}^{l\to\tau}_{\psi,\xi}$), where N_b is a positive integer. The negative samples τ_j and l_j may belong to different domains than (τ_i, l_i). The score function $\mathcal{E}(\tau_i, l)$ is defined as the exponential of the cosine similarity between the encodings, which can be expressed as:

$$\mathcal{E}(\tau_i, l) = \exp\Big(\langle f_{\psi_i}(\tau_i), f_\xi(l)\rangle\Big). \tag{5}$$

where f_{ψ_i} represents the trajectory encoder for the domain to which trajectory τ_i belongs. The operator $\langle\cdot,\cdot\rangle$ denotes the cosine similarity between the two inputs.

Associating Motions Based on the Intention Similarity Score

When robot i receives a demonstration τ_j from robot j for imitation, robot i replicates τ_j by associating τ_j with one of its executable motions. The association process involves the following steps. First, given the current state s_i, a batch of candidate trajectories $\{\tau_i^{(k)}\}$ from the motion generator f_{θ_i} of robot i is sampled. Each sampled trajectory $\tau_i^{(k)} = (s_i, \mathbf{a}_i^{(k)})$ is then encoded by its motion encoder $f_{\psi_i}(\tau_i^{(k)})$, and the demonstration τ_j is encoded by robot j's motion encoder $f_{\psi_j}(\tau_j)$.

Next, each sampled trajectory for robot i is evaluated based on its validity as being in-distribution and the similarity of its intention to that of τ_j. The validity is calculated as

$$V_{valid}(\tau_i^{(k)}) = -\mathcal{E}(\tau_i^{(k)}, \text{“unknown”}) = -\exp\Big(\langle f_{\psi_i}(\tau_i^{(k)}), f_\xi(\text{“unknown”})\rangle\Big), \tag{6}$$

which represents the encoding distance between the given trajectory and the “unknown” label. The intention similarity is calculated as

$$V_{aligned}(\tau_i^{(k)}, \tau_j) = \mathcal{E}(\tau_i^{(k)}, \tau_j) = \exp\Big(\langle f_{\psi_i}(\tau_i^{(k)}), f_{\psi_j}(\tau_j)\rangle\Big)., \tag{7}$$

which represents the encoding distance between the given trajectory and the demonstration τ_j. We use two preset thresholds, ϵ_{valid} and $\epsilon_{aligned}$, to define the minimum requirements for the in-distribution validity and intention alignment, respectively. The candidate batch $\mathcal{B}$ containing valid actions to replicate τ_j is then defined as follows:

$$\mathcal{B} = \left\{ \tau_i^{(k)} \mid \tau_i^{(k)} \sim p_{\theta_i},\ V_{aligned}(\tau_i^{(k)}, \tau_j) > \epsilon_{aligned},\ V_{valid}(\tau_i^{(k)}) > \epsilon_{valid} \right\}. \tag{8}$$

where p_{θ_i} denotes the motion generator of robot i, and $\tau_i^{(k)} = (s_i, \mathbf{a}_i^{(k)})$ represents the k-th sampled trajectory for robot i.

If no candidate actions satisfy both thresholds, robot i remains inactive, indicating its inability to replicate the demonstrated task. Conversely, if one or more candidate actions meet the criteria, the trajectory with the highest overall score is selected for execution. The action selection policy $\pi(\tau \mid \tau_j)$ is therefore defined as:

$$\pi(\tau \mid \tau_j) = \begin{cases} \arg\max_{\tau_i^{(k)} \in \mathcal{B}} V(\tau_i^{(k)}, \tau_j) & \text{if } |\mathcal{B}| > 0, \\ \text{inactive} & \text{otherwise.} \end{cases} \tag{9}$$

where V is the overall score, which is defined as:

$$V(\tau_i^{(k)}, \tau_j) = V_{aligned}(\tau_i^{(k)}, \tau_j) + V_{valid}(\tau_i^{(k)}), \tau_j) \tag{10}$$

In summary, robot i selects an action that is both valid (in-distribution) and aligned with the intention of the demonstrated action, ensuring safe and effective imitation of motions across different robots.

Assigning motions to the best robot on the team

When a team of robots receives a demonstration τ_j by robot j, we extend the action association process to leverage the capabilities of all available robots within the team. Instead of sampling actions from a single robot, we draw candidate trajectories from all learner robots on the team. This approach enhances the flexibility and adaptability of the system by utilizing the diverse motion capabilities of multiple robots.

This flexibility is achieved by incorporating sample trajectories from all available robots in the selection process, as outlined in Eq. (11). We define a set of robots, denoted as $\mathcal{R}_{team}$, representing the available target robots within the team. The extended candidate batch $\mathcal{B}_{team}$ is constructed as follows:

$$\mathcal{B}_{team} = \bigcup_{i \in \mathcal{R}_{team}} \left\{ \tau_i^{(k)} \mid \tau_i^{(k)} \sim p_{\theta_i},\ V_{valid}(\tau_i^{(k)}) > \epsilon_{valid},\ V_{aligned}(\tau_i^{(k)}, \tau_j) > \epsilon_{aligned} \right\}, \tag{11}$$

where p_{θ_i} denotes the motion generator of robot i on the team, and $\tau_i^{(k)} = (s_i, \mathbf{a}_i^{(k)})$ represents the k-th sampled trajectory for robot i. Here, s_i is the current state of robot i, and $\mathbf{a}_i^{(k)}$ is the action sequence generated by $p_{\theta_i}(s_i, z)$ with $z \sim p(z)$.

The action selection criteria are the same as those in Eq. (9). If no candidate trajectories satisfy both thresholds, the robot team remains inactive, indicating that no robot on the team is able to replicate the demonstrated task. Conversely, if one or more candidates meet the criteria, the trajectory with the highest overall score $V = V_{aligned} + V_{valid}$ is selected for execution by the robot that generated this action. This procedure is illustrated in Fig. 3.

Statistical analysis

Monitoring task

We use Welch's unequal-variance t-test for the monitoring task. Test statistics for the 16 demonstrator–learner pairs for comparing IAIL against the density-based baseline and the description-based baseline are reported in table S3 and table S4, respectively. The statistics are computed using 1,500 test scores for each method. For each pair, we report the mean difference Δ (IAIL − baseline), its 95% confidence interval (CI), Welch's t statistic, degrees of freedom (df), two-sided p, and the effect size Cohen's d (with 95% CI). All values are shown with two decimals, p values smaller than 0.001 are shown as "< 0.001".

When scores are nearly deterministic (for example, all 1,500 tests have identical scores) for both methods, the within-condition standard deviations become extremely small. The corresponding t, df, and Cohen's d are ill-defined or numerically unstable. For such rows, we indicate t, df, and Cohen's d as "–". The eight pairs with substantial distributional divergence and the four pairs with capability mismatch are highlighted in bold in table S3 and table S4, respectively.

Item picking task

We use Welch's unequal-variance t-test for the item picking task. Test statistics for the 9 demonstrator–learner pairs for comparing IAIL against the density-based baseline and the description-based baseline are reported in table S5 and table S6, respectively. For each pair we report the mean difference Δ (IAIL − baseline), its 95% confidence interval (CI), Welch's t statistic, degrees of freedom (df), two-sided p, and the effect size Cohen's d (with 95% CI). Following standard practice, p values smaller than 0.001 are shown as "< 0.001".

Supplementary Materials

Supplementary Methods
Supplementary Experimental Setups

Supplementary Results
Figs. S1 to S5
Tables S1 to S6
Movies S1 to S3

References and Notes

1. B. D. Argall, S. Chernova, M. Veloso, B. Browning, A survey of robot learning from demonstration. *Robot. Auton. Syst.* **57** (5), 469–483 (2009).

2. H. Ravichandar, A. S. Polydoros, S. Chernova, A. Billard, Recent advances in robot learning from demonstration. *Annu. Rev. Control Robot. Auton. Syst.* **3**, 297–330 (2020).

3. Y. Ma, A. Cramariuc, F. Farshidian, M. Hutter, Learning coordinated badminton skills for legged manipulators. *Sci. Robot.* **10** (102), eadu3922 (2025), doi:10.1126/scirobotics.adu3922.

4. X. Chen, A. Ghadirzadeh, T. Yu, J. Wang, A. Y. Gao, W. Li, L. Bin, C. Finn, C. Zhang, Lapo: Latent-variable advantage-weighted policy optimization for offline reinforcement learning. *Adv. Neural Inf. Process. Syst. (NeurIPS)* **35**, 36902–36913 (2022).

5. M. Zare, P. M. Kebria, A. Khosravi, S. Nahavandi, A Survey of Imitation Learning: Algorithms, Recent Developments, and Challenges. *IEEE Trans. Cybern.* **54** (12), 7173–7186 (2024), doi: 10.1109/TCYB.2024.3395626.

6. Y. Hu, F. J. Abu-Dakka, F. Chen, X. Luo, Z. Li, A. Knoll, W. Ding, Fusion dynamical systems with machine learning in imitation learning: A comprehensive overview. *Information Fusion* **108**, 102379 (2024), doi:https://doi.org/10.1016/j.inffus.2024.102379, `https://www.sciencedirect.com/science/article/pii/S156625352400157X`.

7. A. Ghadirzadeh, X. Chen, P. Poklukar, C. Finn, M. Björkman, D. Kragic, Bayesian meta-learning for few-shot policy adaptation across robotic platforms, in *IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)* (IEEE) (2021), pp. 1274–1280.

8. D. Hejna, L. Pinto, P. Abbeel, Hierarchically decoupled imitation for morphological transfer, in *Proc. Int. Conf. Mach. Learn. (ICML)* (2020), pp. 4159–4171.

9. Y. Liu, A. Gupta, P. Abbeel, S. Levine, Imitation from observation: Learning to imitate behaviors from raw video via context translation, in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)* (IEEE) (2018), pp. 1118–1125.

10. H. Liu, C. Zhang, Y. Zhu, C. Jiang, S.-C. Zhu, Mirroring without overimitation: Learning functionally equivalent manipulation actions, in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 33 (2019), pp. 8025–8033.

11. X. Chen, A. Ghadirzadeh, M. Björkman, P. Jensfelt, Adversarial feature training for generalizable robotic visuomotor control, in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)* (IEEE) (2020), pp. 1142–1148.

12. J. Lee, M. Bjelonic, A. Reske, L. Wellhausen, T. Miki, M. Hutter, Learning robust autonomous navigation and locomotion for wheeled-legged robots. *Sci. Robot.* **9** (89), eadi9641 (2024).

13. C. Finn, T. Yu, T. Zhang, P. Abbeel, S. Levine, One-shot visual imitation learning via meta-learning, in *Proc. Conf. Robot. Learn. (CoRL)* (2017), pp. 357–368.

14. T. Yu, C. Finn, S. Dasari, A. Xie, T. Zhang, P. Abbeel, S. Levine, One-Shot Imitation from Observing Humans via Domain-Adaptive Meta-Learning, in *Proc. Robot.: Sci. Syst. (RSS)* (Pittsburgh, Pennsylvania) (2018), doi:10.15607/RSS.2018.XIV.002.

15. E. Jang, A. Irpan, M. Khansari, D. Kappler, F. Ebert, C. Lynch, S. Levine, C. Finn, Bc-z: Zero-shot task generalization with robotic imitation learning, in *Proc. Conf. Robot. Learn. (CoRL)* (2022), pp. 991–1002.

16. J. Beck, R. Vuorio, E. Z. Liu, Z. Xiong, L. Zintgraf, C. Finn, S. Whiteson, A tutorial on meta-reinforcement learning. *Found. Trends Mach. Learn.* **18** (2-3), 224–384 (2025).

17. J. Li, T. Lu, X. Cao, Y. Cai, S. Wang, Meta-Imitation Learning by Watching Video Demonstrations, in *Int. Conf. Learn. Represent. (ICLR)* (2021).

18. T. Shankar, Y. Lin, A. Rajeswaran, V. Kumar, S. Anderson, J. Oh, Translating Robot Skills: Learning Unsupervised Skill Correspondences Across Robots, in *Proc. Int. Conf. Mach. Learn. (ICML)* (2022), pp. 19626–19644.

19. M. Bauza, A. Bronars, Y. Hou, I. Taylor, N. Chavan-Dafle, A. Rodriguez, SimPLE, a visuotactile method learned in simulation to precisely pick, localize, regrasp, and place objects. *Sci. Robot.* **9** (91), eadi8808 (2024).

20. J. Allen, J. Anderson, J. Baltes, Vision-based imitation learning in heterogeneous multi-robot systems: Varying physiology and skill. *Int. J. Autom. Smart Technol.* **2** (2), 147–161 (2012).

21. J. Song, H. Ren, D. Sadigh, S. Ermon, Multi-agent generative adversarial imitation learning, in *Adv. Neural Inf. Process. Syst. (NeurIPS)* (2018), pp. 7472–7483.

22. P. Feng, T. Yang, M. Liang, L. Wang, Y. Gao, OC-HMAS: Dynamic Self-Organization and Self-Correction in Heterogeneous Multiagent Systems Using Multimodal Large Models. *IEEE Internet Things J.* **12** (10), 13538–13555 (2025), doi:10.1109/JIOT.2025.3545496.

23. Y. Gao, J. Chen, X. Chen, C. Wang, J. Hu, F. Deng, T. L. Lam, Asymmetric Self-Play-Enabled Intelligent Heterogeneous Multirobot Catching System Using Deep Multiagent Reinforcement Learning. *IEEE Trans. Robot.* **39** (4), 2603–2622 (2023), doi:10.1109/TRO.2023.3257541.

24. H. Chakraa, F. Guérin, E. Leclercq, D. Lefebvre, Optimization techniques for Multi-Robot Task Allocation problems: Review on the state-of-the-art. *Robot. Auton. Syst.* **168** (2023), doi:10.1016/j.robot.2023.104492, https://doi.org/10.1016/j.robot.2023.104492.

25. K. Athira, S. Umashankar, A Systematic Literature Review on Multi-Robot Task Allocation. *ACM Comput. Surv.* **57** (3), 1–28 (2024).

26. G. Gergely, H. Bekkering, I. Király, Rational imitation in preverbal infants. *Nature* **415** (6873), 755–755 (2002).

27. A. N. Meltzoff, Understanding the intentions of others: re-enactment of intended acts by 18-month-old children. *Dev. Psychol.* **31** (5), 838–850 (1995).

28. M. Tomasello, Cultural learning redux. *Child Dev.* **87** (3), 643–653 (2016).

29. B. S. Hewlett, A. H. Boyette, S. Lew-Levy, S. Gallois, S. J. Dira, Cultural transmission among hunter-gatherers. *Proc. Natl. Acad. Sci. U.S.A.* **121** (48), e2322883121 (2024).

30. Z. H. Garfield, S. Lew-Levy, Teaching is associated with the transmission of opaque culture and leadership across 23 egalitarian hunter-gatherer societies. *Nat. Commun.* **16** (1), 3387 (2025).

31. L. Bonini, C. Rotunno, E. Arcuri, V. Gallese, Mirror neurons 30 years later: implications and applications. *Trends Cogn. Sci.* **26** (9), 767–781 (2022).

32. E. Oztop, M. Kawato, M. A. Arbib, Mirror neurons: functions, mechanisms and models. *Neurosci. Lett.* **540**, 43–55 (2013).

33. E. Oztop, M. Kawato, M. Arbib, Mirror neurons and imitation: A computationally guided review. *Neural Netw.* **19** (3), 254–271 (2006).

34. N. Nishitani, R. Hari, Temporal dynamics of cortical representation for action. *Proc. Natl. Acad. Sci. U.S.A.* **97** (2), 913–918 (2000).

35. J. Fischer, Physical reasoning is the missing link between action goals and kinematics. A comment on" An active inference model of hierarchical action understanding, learning, and imitation" by Proietti et al. *Phys. Life Rev.* **48**, 198–200 (2024).

36. Ryze Tech, TELLO User Manual v1.4, https://dl-cdn.ryzerobotics.com/downloads/Tello/Tello%20User%20Manual%20v1.4.pdf (2018), accessed 11 Feb 2026.

37. Shenzhen Moying Technology Co., Ltd., Robot product information, https://moyingrobotics.com/en/Robot/index.html, accessed 11 Feb 2026.

38. NXROBO, spark_noetic: ROS wrapper and applications for the Spark robot (GitHub repository), https://github.com/NXROBO/spark_noetic, accessed 11 Feb 2026.

39. L. Zhang, Y. Huang, Z. Cao, Y. Jiao, H. Qian, Parallel Self-Assembly for a Multi-USV System on Water Surface With Obstacles. *IEEE Trans. Autom. Sci. Eng.* **22**, 2213–2224 (2025), doi: 10.1109/TASE.2024.3376981.

40. SoftBank Robotics America, Meet Pepper: The robot built for people, https://us.softbankrobotics.com/pepper, accessed 11 Feb 2026.

41. Direct Drive Technology Limited, DIABLO: Product page, https://en.directdrive.com/product_diablo, accessed 11 Feb 2026.

42. D. L. Davies, D. W. Bouldin, A Cluster Separation Measure. *IEEE Trans. Pattern Anal. Mach. Intell. (PAMI)* **1** (2), 224–227 (1979), doi:10.1109/TPAMI.1979.4766909.

43. K. Hu, Z. Rui, Y. He, Y. Liu, P. Hua, H. Xu, Stem-OB: Generalizable Visual Imitation Learning with Stem-Like Convergent Observation through Diffusion Inversion, in *Int. Conf. Learn. Represent. (ICLR)* (2025).

44. K. Pertsch, R. Desai, V. Kumar, F. Meier, J. J. Lim, D. Batra, A. Rai, Cross-Domain Transfer via Semantic Skill Imitation, in *Proc. Conf. Robot. Learn. (CoRL)* (2023), pp. 690–700.

45. L. Y. Chen, C. Xu, K. Dharmarajan, R. Cheng, K. Keutzer, M. Tomizuka, Q. Vuong, K. Goldberg, RoVi-Aug: Robot and Viewpoint Augmentation for Cross-Embodiment Robot Learning, in *Proc. Conf. Robot. Learn. (CoRL)* (2025), pp. 209–233.

46. I. Nematollahi, B. DeMoss, A. L. Chandra, N. Hawes, W. Burgard, I. Posner, LUMOS: Language-Conditioned Imitation Learning with World Models, in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)* (2025), pp. 8219–8225, doi:10.1109/ICRA55743.2025.11127988.

47. X. Yao, T. Blei, Y. Meng, Y. Zhang, H. Zhou, Z. Bing, K. Huang, F. Sun, A. Knoll, Long-Horizon Language-Conditioned Imitation Learning for Robotic Manipulation. *IEEE/ASME Trans. Mechatronics* **30** (6), 5628–5639 (2025), doi:10.1109/TMECH.2025.3547047.

48. B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, Q. Vuong, V. Vanhoucke, H. Tran, R. Soricut, A. Singh, J. Singh, P. Sermanet, P. R. Sanketi, G. Salazar, M. S. Ryoo, K. Reymann, K. Rao, K. Pertsch, I. Mordatch, H. Michalewski, Y. Lu, S. Levine, L. Lee, T.-W. E. Lee, I. Leal, Y. Kuang, D. Kalashnikov, R. Julian, N. J. Joshi, A. Irpan, B. Ichter, J. Hsu, A. Herzog, K. Hausman, K. Gopalakrishnan, C. Fu, P. Florence, C. Finn, K. A. Dubey, D. Driess, T. Ding, K. M. Choromanski, X. Chen, Y. Chebotar, J. Carbajal, N. Brown, A. Brohan, M. G. Arenas, K. Han, RT-2: Vision-Language-Action Models Transfer

Web Knowledge to Robotic Control, in *Proc. Conf. Robot. Learn. (CoRL)*, J. Tan, M. Toussaint, K. Darvish, Eds., vol. 229 of *Proceedings of Machine Learning Research* (2023), pp. 2165–2183, https://proceedings.mlr.press/v229/zitkovich23a.html.

49. H. Zhou, Z. Bing, X. Yao, X. Su, C. Yang, K. Huang, A. Knoll, Language-Conditioned Imitation Learning With Base Skill Priors Under Unstructured Data. *IEEE Robot. Autom. Lett.* **9** (11), 9805–9812 (2024), doi:10.1109/LRA.2024.3466076.

50. A. O'Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, A. Tung, A. Bewley, A. Herzog, A. Irpan, A. Khazatsky, A. Rai, A. Gupta, A. Wang, A. Singh, A. Garg, A. Kembhavi, A. Xie, A. Brohan, A. Raffin, A. Sharma, A. Yavary, A. Jain, A. Balakrishna, A. Wahid, B. Burgess-Limerick, B. Kim, B. Schölkopf, B. Wulfe, B. Ichter, C. Lu, C. Xu, C. Le, C. Finn, C. Wang, C. Xu, C. Chi, C. Huang, C. Chan, C. Agia, C. Pan, C. Fu, C. Devin, D. Xu, D. Morton, D. Driess, D. Chen, D. Pathak, D. Shah, D. Büchler, D. Jayaraman, D. Kalashnikov, D. Sadigh, E. Johns, E. Foster, F. Liu, F. Ceola, F. Xia, F. Zhao, F. Stulp, G. Zhou, G. S. Sukhatme, G. Salhotra, G. Yan, G. Feng, G. Schiavi, G. Berseth, G. Kahn, G. Wang, H. Su, H.-S. Fang, H. Shi, H. Bao, H. Ben Amor, H. I. Christensen, H. Furuta, H. Walke, H. Fang, H. Ha, I. Mordatch, I. Radosavovic, I. Leal, J. Liang, J. Abou-Chakra, J. Kim, J. Drake, J. Peters, J. Schneider, J. Hsu, J. Bohg, J. Bingham, J. Wu, J. Gao, J. Hu, J. Wu, J. Wu, J. Sun, J. Luo, J. Gu, J. Tan, J. Oh, J. Wu, J. Lu, J. Yang, J. Malik, J. Silvério, J. Hejna, J. Booher, J. Tompson, J. Yang, J. Salvador, J. J. Lim, J. Han, K. Wang, K. Rao, K. Pertsch, K. Hausman, K. Go, K. Gopalakrishnan, K. Goldberg, K. Byrne, K. Oslund, K. Kawaharazuka, K. Black, K. Lin, K. Zhang, K. Ehsani, K. Lekkala, K. Ellis, K. Rana, K. Srinivasan, K. Fang, K. P. Singh, K.-H. Zeng, K. Hatch, K. Hsu, L. Itti, L. Y. Chen, L. Pinto, L. Fei-Fei, L. Tan, L. J. Fan, L. Ott, L. Lee, L. Weihs, M. Chen, M. Lepert, M. Memmel, M. Tomizuka, M. Itkina, M. G. Castro, M. Spero, M. Du, M. Ahn, M. C. Yip, M. Zhang, M. Ding, M. Heo, M. K. Srirama, M. Sharma, M. J. Kim, N. Kanazawa, N. Hansen, N. Heess, N. J. Joshi, N. Suenderhauf, N. Liu, N. Di Palo, N. M. M. Shafiullah, O. Mees, O. Kroemer, O. Bastani, P. R. Sanketi, P. T. Miller, P. Yin, P. Wohlhart, P. Xu, P. D. Fagan, P. Mitrano, P. Sermanet, P. Abbeel, P. Sundaresan, Q. Chen, Q. Vuong, R. Rafailov, R. Tian, R. Doshi, R. Martín-Martín, R. Baijal, R. Scalise, R. Hendrix, R. Lin, R. Qian, R. Zhang, R. Mendonca, R. Shah, R. Hoque, R. Julian, S. Bustamante, S. Kirmani, S. Levine, S. Lin, S. Moore, S. Bahl, S. Dass, S. Sonawani, S. Song, S. Xu, S. Haldar, S. Karamcheti, S. Adebola, S. Guist, S. Nasiriany, S. Schaal, S. Welker, S. Tian, S. Ramamoorthy, S. Dasari, S. Belkhale, S. Park, S. Nair, S. Mirchandani, T. Osa, T. Gupta, T. Harada, T. Matsushima, T. Xiao, T. Kollar, T. Yu, T. Ding, T. Davchev, T. Z. Zhao, T. Armstrong, T. Darrell, T. Chung, V. Jain, V. Vanhoucke, W. Zhan, W. Zhou, W. Burgard, X. Chen, X. Wang, X. Zhu, X. Geng, X. Liu, X. Liangwei, X. Li, Y. Lu, Y. J. Ma, Y. Kim, Y. Chebotar, Y. Zhou, Y. Zhu, Y. Wu,

Y. Xu, Y. Wang, Y. Bisk, Y. Cho, Y. Lee, Y. Cui, Y. Cao, Y.-H. Wu, Y. Tang, Y. Zhu, Y. Zhang, Y. Jiang, Y. Li, Y. Li, Y. Iwasawa, Y. Matsuo, Z. Ma, Z. Xu, Z. J. Cui, Z. Zhang, Z. Lin, Open X-Embodiment: Robotic Learning Datasets and RT-X Models : Open X-Embodiment Collaboration, in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)* (2024), pp. 6892–6903, doi: 10.1109/ICRA57147.2024.10611477.

51. Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, C. Xu, J. Luo, T. Kreiman, Y. Tan, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, S. Levine, Octo: An Open-Source Generalist Robot Policy, in *Proc. Robot.: Sci. Syst. (RSS)* (Delft, Netherlands) (2024).

52. M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, C. Finn, OpenVLA: An Open-Source Vision-Language-Action Model, in *Proc. Conf. Robot Learn. (CoRL)* (2024), pp. 1–22.

53. A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, I. Sutskever, Learning Transferable Visual Models From Natural Language Supervision, in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 139 of *Proceedings of Machine Learning Research* (2021), pp. 8748–8763, https://proceedings.mlr.press/v139/radford21a.html.

54. University Robots, UR5 Technical specifications, https://www.universal-robots.com/media/50588/ur5_en.pdf, accessed 11 Feb 2026.

55. L. Wang, X. Chen, J. Zhao, K. He, Scaling Proprioceptive-Visual Learning with Heterogeneous Pre-trained Transformers, in *Adv. Neural Inf. Process. Syst. (NeurIPS)* (Curran Associates, Inc.), vol. 37 (2024), pp. 124420–124450, doi:10.52202/079017-3952, https://proceedings.neurips.cc/paper_files/paper/2024/file/e0f393e7980a24fd12fa6f15adfa25fb-Paper-Conference.pdf.

56. D. He, C. Fang, Y. Wang, Y. Peng, Y. Wang, S.-C. Zhu, A Mathematical Formulation of AGI in the (C, U, V) Framework. *Engineering* (2025), doi:https://doi.org/10.1016/j.eng.2025.08.034, https://www.sciencedirect.com/science/article/pii/S2095809925005521.

57. P. Wei, Y. Liu, T. Shu, N. Zheng, S.-C. Zhu, Where and why are they looking? jointly inferring human attention and intentions in complex tasks, in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)* (2018), pp. 6801–6809.

58. C. L. Baker, R. Saxe, J. B. Tenenbaum, Action understanding as inverse planning. *Cognition* **113** (3), 329–349 (2009).

59. J. A. Sommerville, A. L. Woodward, A. Needham, Action experience alters 3-month-old infants' perception of others' actions. *Cognition* **96** (1), B1–B11 (2005).

60. A. D. Dragan, K. C. Lee, S. S. Srinivasa, Legibility and predictability of robot motion, in *ACM/IEEE Int. Conf. Hum.-Robot Interact. (HRI)* (2013), pp. 301–308.

61. M. Zurek, A. Bobu, D. S. Brown, A. D. Dragan, Situational confidence assistance for lifelong shared autonomy, in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)* (IEEE) (2021), pp. 2783–2789.

62. C. Liu, J. B. Hamrick, J. F. Fisac, A. D. Dragan, J. K. Hedrick, S. S. Sastry, T. L. Griffiths, Goal Inference Improves Objective and Perceived Performance in Human-Robot Collaboration, in *Proc. Int. Conf. Auton. Agents Multiagent Syst. (AAMAS)* (2016), pp. 940–948.

63. K. M. Collins, I. Sucholutsky, U. Bhatt, K. Chandra, L. Wong, M. Lee, C. E. Zhang, T. Zhi-Xuan, M. Ho, V. Mansinghka, A. Weller, J. B. Tenenbaum, T. L. Griffiths, Building machines that learn and think with people. *Nat. Hum. Behav.* **8** (10), 1851–1863 (2024).

64. I. Higgins, L. Matthey, A. Pal, C. P. Burgess, X. Glorot, M. M. Botvinick, S. Mohamed, A. Lerchner, beta-vae: Learning basic visual concepts with a constrained variational framework. *Int. Conf. Learn. Represent. (ICLR)* (2017).

65. I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial networks. *Commun. ACM* **63** (11), 139–144 (2020).

66. C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, S. Song, Diffusion policy: Visuomotor policy learning via action diffusion. *Int. J. Robot. Res.* **44** (10-11), 1684–1704 (2025).

67. A. Salmona, V. De Bortoli, J. Delon, A. Desolneux, Can Push-forward Generative Models Fit Multimodal Distributions? *Adv. Neural Inf. Process. Syst. (NeurIPS)* **35**, 10766–10779 (2022).

68. R. Bellman, A Markovian Decision Process. *J. Math. Mech.* **6** (5), 679–684 (1957), `http://www.jstor.org/stable/24900506`.

69. M. Shridhar, L. Manuelli, D. Fox, Cliport: What and where pathways for robotic manipulation, in *Proc. Conf. Robot. Learn. (CoRL)* (2022), pp. 894–906.

70. L. Downs, A. Francis, N. Koenig, B. Kinman, R. Hickman, K. Reymann, T. B. McHugh, V. Vanhoucke, Google scanned objects: A high-quality dataset of 3d scanned household items, in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)* (2022), pp. 2553–2560.

71. E. Perez, F. Strub, H. de Vries, V. Dumoulin, A. Courville, FiLM: visual reasoning with a general conditioning layer, in *Proc. AAAI Conf. Artif. Intell.* (AAAI Press) (2018), pp. 3942–3951.

72. S. H. Vemprala, R. Bonatti, A. Bucker, A. Kapoor, Chatgpt for robotics: Design principles and model abilities. *IEEE Access* **12**, 55682–55696 (2024).

73. P. Vijayaraghavan, J. F. Queißer, S. V. Flores, J. Tani, Development of compositionality through interactive learning of language and action of robots. *Sci. Robot.* **10** (98), eadp0751 (2025).

Acknowledgments

The first authors thank their beloved son H. Gao for his support during the preparation of this manuscript.

Funding: This work was supported by the National Key R&D Program of China (Grant Nos. 2024YFB4505500 and 2024YFB4505503), in part by the National Natural Science Foundation of China (Grant No. 62376031), the Shenzhen Science and Technology Program (Grant No. JSGGKQTD20221101115656029 and ZDCY20250901104706008), and Guangdong Basic and Applied Basic Research Foundation (Grant number: 2023B1515020089).

Author contributions: X.C. conceived the methodology, designed and performed the simulation and real-world experiments, wrote the manuscript and partially supervised the project. Y.G. defined the problem conceptualization and contributed to methodology discussions, designed and performed the real-world experiments, wrote the manuscript and provided partial funding. H.L. and F.Y. contributed to the discussions and writing of the manuscript. A.G. contributed to the technical discussions. J.Y., B.L., and S.-C.Z. acquired funding and resources. C.Z. and T.L.L. contributed to the technical discussions and partially supervised the project.

Competing interests: The authors declare that they have no competing interests.

Data, Code and Materials availability: All data needed to evaluate the conclusions in the paper are present in the paper or the Supplementary Materials. The evaluation scripts used in this study are available at: `https://zenodo.org/records/18618978` and `https://github.com/pcchenxi/Cross-Robot-Behavior-Adaptation-through-Intention-Alignment`

Supplementary Materials for Cross-Robot Behavior Adaptation through Intention Alignment

Xi Chen*†, Yuan Gao†, Chongjie Zhang*, Tin Lun Lam*
*Corresponding authors. Emails: chenxi@bigai.ai, gaoyuan@cuhk.edu.cn, czhang@wustl.edu, tllam@cuhk.edu.cn
†These authors contributed equally to this work and are listed alphabetically.

This PDF file includes:

Supplementary Methods
Supplementary Experimental Setups
Supplementary Results
Figs. S1 to S5
Tables S1 to S6
Captions for Movies S1 to S3

Other Supplementary Materials for this manuscript:

Movies S1 to S3

Supplementary Methods

Problem Statement

Markov Decision Process We model our sequential decision-making task as a finite Markov decision process (MDP) (*68*), defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \rho_0, \gamma)$. Here, $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, and $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the transition function, where $P(s'|s, a)$ represents the probability of transitioning to the next state by performing action a in state s. $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function, ρ_0 is the distribution of the initial state, and $\gamma \in [0, 1]$ is a discount factor. A policy $\pi : \mathcal{S} \to \mathcal{A}$ is a mapping from states to actions, with $\pi(a|s)$ specifying the probability of performing action a in state s. In each episode, an initial state s_0 is sampled from ρ_0 by the environment and sent to the agent. The agent receives the state and generates an action a_0 from policy $\pi(a|s)$. Then, the action is executed in the environment, and the next state is entered according to the probability transition function $P(s'|s, a)$. A numerical reward r_0 is also produced based on the reward function $R(s, a)$. This interactive process is repeated H times before the episode is terminated. The sequence of the acquired state-action tuples is a trajectory denoted as $\tau = \{(s_t, a_t)\}_{t=0}^{H}$.

The discounted accumulative reward, also known as the return of trajectory τ, is denoted as $G(\tau) = \sum_{t=0}^{H} \gamma^t R(s_t, a_t)$. The performance or value of a policy is evaluated as the expected return of the trajectories sampled from the policy, which is computed as $\mathbb{E}_{\tau \sim p(\pi)}[G(\tau)]$, where $p(\pi)$ represents the distribution of trajectories sampled from policy π. The policy is an optimal policy if the expected return is maximized.

Definition of domain In our work, the concept of a domain is defined as a multitask MDP comprising a robot, an environment, and a set of tasks that the robot can perform within this environment. We represent a domain as a 7-element tuple $M = (\mathcal{S}, \mathcal{A}, P, R, \rho_0, \mu, \gamma)$, where μ is the task set, which belongs to a global task space $\mathcal{K}$. Here, the task is included as an additional input to the reward function, denoted as $R(s, a, k) : \mathcal{S} \times \mathcal{A} \times \mathcal{K} \to \mathbb{R}$. Thus, different rewards may be received for the same state-action pair (s, a) depending on the task the agent is attempting to complete. Accordingly, the return of a trajectory is also task dependent, which is computed as:

$$G(\tau, k) = \sum_{t=0}^{H} \gamma^t R(s_t, a_t, k). \tag{S1}$$

We assume that, for any task $k \in \mu$, there exist optimal trajectories with returns exceeding a threshold $G_{complete}$, indicating that tasks sampled from μ can be fully achieved within the domain.

Imitation learning with heterogeneous robots We consider a set of heterogeneous robot domains $\mathcal{M} = \{M_i\}_{i=1}^{N}$, where each domain in the set varies in some aspects. The goal is to enable

robots within this set to complete various tasks by imitating behaviors of other robots within the set. We refer to the domain in which the demonstration is provided as the source domain M_{src} and the domain in which the demonstration is imitated as the target domain M_{trg}. The robot in the source domain is referred to as the source robot or the demonstrator. Similarly, the robot in the target domain is referred to as the target robot or the learner. Importantly, any robot in the set $\mathcal{M}$ can be a source or target robot in different learning scenarios.

We follow a one-shot IL setting, where, in each run, a source robot from M_{src} provides a single demonstration of a task to a target robot in M_{trg}. The target robot must then generate an action trajectory that maximizes the return for the same task. Specifically, our goal is to learn a demonstration-conditioned policy $\pi(\tau_{trg}|\tau_{src})$ such that, for any given pair of source and target robots M_{src} and M_{trg} in $\mathcal{M}$, the policy generates a trajectory in the target domain that maximizes the return of the task demonstrated. The policy can be formulated as follows:

$$\pi(\tau_{trg}|\tau_{src}) = \arg\max_{\pi} \mathop{\mathbb{E}}_{\substack{k\sim p_{src}(k|\tau_{src}) \\ \tau_{trg}\sim\pi}} \Big[G_{trg}(\tau_{trg}, k) \Big], \tag{S2}$$

where τ_{src} is the demonstration provided by the source robot, $p_{src}(k|\tau_{src})$ represents the task probability of the demonstration in the source domain, and $G_{trg}(\tau_{trg}, k)$ is the return of the robot in the target domain for completing task k.

However, optimizing this objective requires a cross-domain performance evaluation (evaluating a source domain task in the target domain), which is difficult in many practical scenarios, as sufficient knowledge of the demonstrated tasks and the reward function in both domains is needed. Thus, in this work, we propose extracting the high-level intention of the trajectory and evaluating the trajectory by directly comparing its intention to that of the demonstration. The objective of the policy is expressed as:

$$\pi(\tau_{trg}|\tau_{src}) = \arg\max_{\pi} \mathop{\mathbb{E}}_{\tau_{trg}\sim\pi} \Big[V(\tau_{trg}, \tau_{src}) \Big], \tag{S3}$$

where $V(\tau_{trg}, \tau_{src})$ represents the intention similarity of the two trajectories. The similarity score V is defined in eq. (10), and the calculation procedure is detailed in the Section Materials and Methods.

Supplementary Experimental Setups

Real-world robot sensor and motor control

The seven robots used in real-world experiments varied in terms of their sensing modalities, control strategies and motor actuation mechanisms. In the following, we summarize the control interface and motor configuration of each robot.

Cuboat Cuboat is a holonomic unmanned surface vehicle (USV) designed with a cube-shaped body and four thrusters arranged in a "+" configuration, enabling omnidirectional planar movement. It is equipped with a top-mounted RGB webcam (480×640 resolution) for environmental perception. The robot does not include onboard localization; instead, its global (x, y, yaw) position is estimated in real time using an external OptiTrack motion capture system. Each Cuboat receives motion targets in the form of (x, y, yaw) and is controlled by a Raspberry Pi 4B (running ROS), which communicates with an Arduino Mega onboard that performs low-level motor control. The propulsion is provided by four waterproof brushless DC thrusters, and a PID controller is used to track reference trajectories generated from grid-based A* planning. The Cuboat operates in an indoor water pool 6 m × 6 m, and is capable of precise motion with 10 cm translation and 10° angular mean absolute tracking error.

Tello Tello is a lightweight quadrotor drone developed by Ryze Tech in collaboration with DJI. It is equipped with a 5-megapixel RGB camera, which streams video at 720p and 30 fps; to ensure consistency with other platforms, the image stream is resized to 480×640 resolution. The drone does not have onboard localization and instead relies on the same external OptiTrack motion capture system used by other platforms to provide real-time transition and orientation estimates. In our setup, the drone receives velocity commands generated by a PID controller that tracks reference (x, y, z, yaw) targets while maintaining a fixed roll and pitch. The control is performed by a ground-based computer that communicates with the drone over Wi-Fi using the Tello SDK, operating at a control frequency of approximately 10 Hz. The robot is capable of precise motion with 10 cm translation and 10° angular mean absolute tracking error.

Diablo Diablo is a self-balancing wheeled bipedal robot developed by Direct Drive Tech. The robot is equipped with a top-mounted RGB webcam, with its video stream resized to 480×640 resolution to maintain consistency across platforms. For localization, Diablo relies on an external OptiTrack motion capture system, providing real-time (x, y, yaw, h) pose estimates, where h denotes the robot's height. In our setup, Diablo receives target positions defined by (x, y, yaw, h). A PID controller computes the necessary velocity commands to achieve the desired pose. Control commands are transmitted via a ground-based computer over Wi-Fi using the robot's open SDK, operating at a control frequency of approximately 10 Hz. The robot is capable of precise motion with 5 cm translation and 5° angular mean absolute tracking error.

Spark Spark is a compact differential-drive ground robot developed by NXROBO. In our setup, Spark operates without onboard perception sensors, as its tasks do not require visual input. Localization is provided by the external OptiTrack motion capture system, delivering real-time (x, y, yaw) pose estimates. The robot receives target positions defined by (x, y, yaw), and a PID controller

computes the necessary velocity commands to achieve the desired pose. Control commands are transmitted via a ground-based computer over Wi-Fi using the robot's open SDK, operating at a control frequency of approximately 10 Hz. The robot is capable of precise motion with 5 cm translation and 5° angular mean absolute tracking error.

Pepper Pepper is a humanoid robot developed by SoftBank Robotics, equipped with a wheeled base, 20 degrees of freedom, and a multimodal perception system designed for human-robot interaction. In our setup, Pepper uses one of its onboard RGB head cameras to capture visual input, with the image resolution resized to 480×640 for consistency with other platforms. Localization is provided externally via the same OptiTrack motion capture system, which delivers real-time (x, y, yaw) pose estimates. The robot receives target positions in the form of (x, y, yaw), and a PID controller computes velocity commands to guide it toward the desired pose. Commands are sent from a ground-based computer using the NAOqi API over a wireless connection, with control operating at approximately 10 Hz. The robot is capable of precise motion with 5 cm translation and 5° angular mean absolute tracking error. Although Pepper is not designed for direct object manipulation, it is capable of executing predefined full-body animations. In our experiments, we use a custom-designed animation sequence to simulate a picking behavior, wherein the robot mimics collecting items from a table and placing them into the storage compartment located on its chest.

Single-Arm Single-Arm is a stationary 6-DoF robotic manipulator developed by Moying Robotics, equipped with an electrically actuated gripper capable of binary open/close control. In our setup, it is equipped with two RGB webcams, one facing forward and one facing to the left, with video streams resized to 480×640 resolution for consistency across all platforms. The base of the Single-Arm remains fixed during operation. The robot receives target grasping poses defined by $(x, y, z, \text{opening})$, where the opening parameter controls the binary state of the gripper. Motion planning and execution are handled by ROS MoveIt, which generates joint-space trajectories to reach the specified grasp pose. The gripper achieves a translational mean absolute tracking error of 0.5 cm.

Dual-Arm Dual-Arm is a stationary dual-arm collaborative robot developed by Moying Robotics. In our setup, the robot operates without onboard perception sensors, as it operates in a static task environment where localization and object placement remain consistent across trials. The base of the robot is fixed, and both arms share the same hardware configuration as the Single-Arm robot. The system receives two target grasping poses, $(x, y, z, \text{opening})$ for each arm. The target pose for the left arm corresponds to the location of a drawer handle that it is expected to grasp and pull open, while the target pose for the right arm specifies the location of an object to be grasped. Motion planning and execution are handled by ROS MoveIt, which generates joint-space trajectories for both arms. The two actions are executed sequentially: the left arm first opens the drawer, followed

by the right arm grasping the exposed object. The gripper achieves a translational mean absolute tracking error of 0.5 cm.

Real-world task and environment setup

The physical environment is organized around three primary task types, each associated with designated spatial regions and supported by specific robot embodiments:

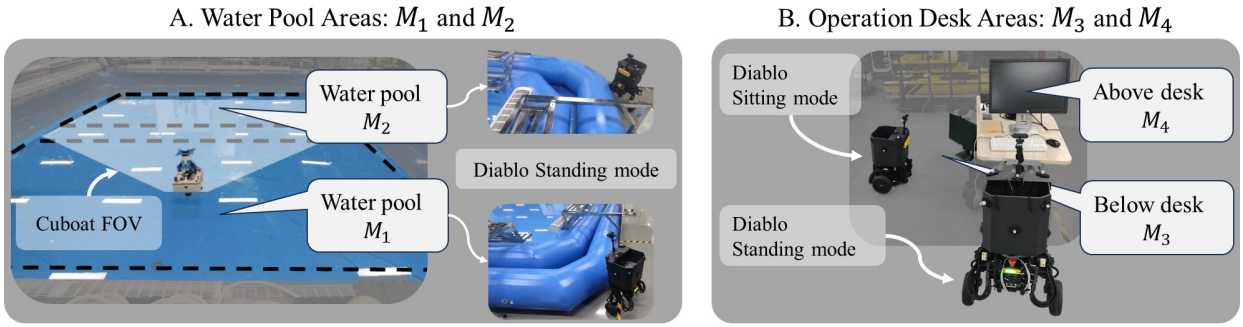

Figure S1: **The environment setup of the user monitoring areas.**

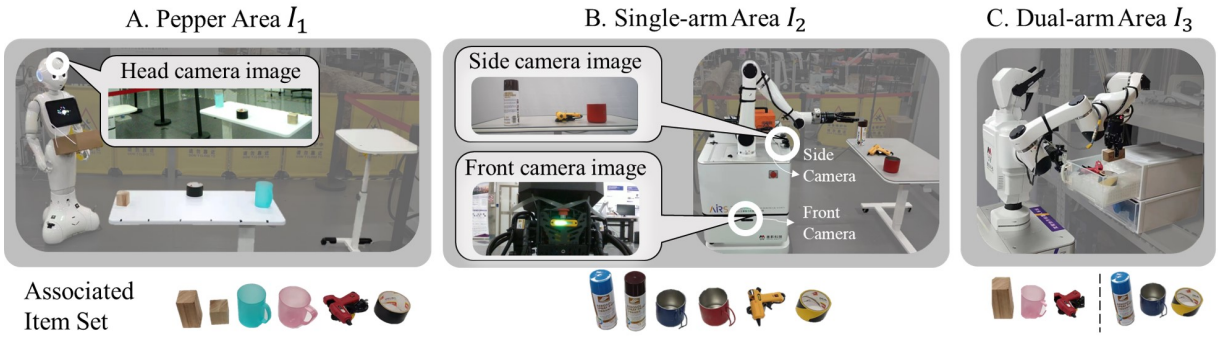

Figure S2: **The environment setup of the item selection areas.**

Figure S3: **The 12 items used in the real-world experiment.** The items are categorized into five classes.

User monitoring locations M_{1-4} These areas simulate surveillance zones where robots are tasked with detecting user activity or intent. Locations M_1 and M_2 are positioned on opposite sides of a 6 m × 6 m indoor water pool. Locations M_3 and M_4 are situated beneath and above a standard workstation adjacent to the pool, corresponding to seated and standing user positions, respectively. The robots capable of performing monitoring tasks include Cuboat (limited to M_1 and M_2), Tello (covering M_1–M_4), and Diablo (covering M_1–M_4). Representative examples of Cuboat and Diablo monitoring these areas are shown in fig. S1. We predefine a set of target monitoring positions and orientations for each robot to monitor each target. The task is considered successful when the robot stops within 15.0,cm and 5° of the target pose.

Item selection locations I_{1-3} These zones involve object retrieval using robotic manipulators. Each zone is associated with a specific robot, with item sets and spatial configurations tailored to its physical capabilities. The setup of the three item selection locations I_{1-3} is visualized in fig. S2.

Specifically, I_1 is a tabletop workspace for Pepper in which three objects are randomly selected from an associated item set, including large/small wooden blocks, blue/pink plastic cups, red glue gun, and black tape. The items are placed with randomized positions and orientations. I_2 is a tabletop workspace for Single-Arm in which three objects are randomly chosen from an associated item set, including blue/brown spray bottles, blue/red metal cups, yellow glue gun, and yellow tape. The items are placed with randomized positions and orientations. I_3 is a two-layer drawer setup for Dual-Arm with six items placed in fixed locations. The top drawer contains a large wooden block, a pink plastic cup, and a red glue gun, while the bottom drawer contains a green spray bottle, a blue metal cup, and black-and-yellow tape."

In total, 12 unique objects are used across all item selection areas. Based on their physical properties and intended functions, these items are grouped into five semantic classes: wood blocks, cups, glue guns, tapes, and paints (fig. S3). The pick up motion is considered successfully when the item is stably grasped by the gripper and lifted up.

Item delivery locations D_{1-2} After an item is retrieved, the manipulation robot transfers it to a transporter robot, which delivers it to one of the designated delivery zones (D_1 or D_2). Robots with delivery capabilities include Pepper, Diablo, and Spark. We predefine a target delivery position and orientation for each location. The task is considered successful when the robot stops within 15.0 cm and 5° of the target pose.

Simulation experiment setup

Target monitoring task The simulation environment for the monitoring task is implemented using NVIDIA Isaac Sim, which provides high-fidelity physics and rendering capabilities. The task involves four robots—Pepper, Wheeled Biped, Cater, and Drone—each equipped with distinct

embodiments and camera configurations. Pepper, Wheeled Biped, and Cater are equipped with front-facing RGB cameras mounted on top of their bodies, while the Drone uses a downward-facing RGB camera mounted beneath its base. All cameras produce noiseless images at a resolution of 640×480 pixels.

The scenario includes two tables and two fixed target boxes. At the beginning of each trial, all robots are placed at central locations between the tables. The geometry and layout of the scene remain constant across trials. A box is considered successfully monitored if it lies within 20 degrees of the center of the robot's camera field of view.

To accommodate embodiment differences, control interfaces vary across robots. Specifically, Pepper is controlled via its planar (x, y) base location, base yaw orientation, and hip pitch angle; Drone via its 3D position (x, y, z); Wheeled Biped via (x, y) position, base yaw orientation, and adjustable body height; and Cater via its (x, y) position and base yaw orientation.

All robots use an RRT-based planner to generate motions. For controlled evaluation, execution is simplified by directly updating base poses rather than simulating joint torques or contact forces. This abstraction allows us to isolate intention alignment effects without confounds from low-level physical noise.

Item picking task The simulation environment for the item picking task is adapted from the pick-and-place benchmark introduced in CLIPort (*69*) and is implemented using the PyBullet physics engine. We simulate three UR5 robotic arms with identical kinematics but different camera placements to introduce embodiment variation. Each robot is positioned in front of a rectangular table and equipped with a noiseless RGB camera (640×480 resolution). The cameras are mounted at distinct locations: in front of the robot (`UR5_F`), on the left shoulder (`UR5_L`), and on the right shoulder (`UR5_R`).

Each robot faces a rectangular table. In every episode, four objects are randomly sampled from a predefined item list associated with the specific robot. The full item pool includes 18 objects drawn from the Google Scanned Objects dataset (*70*), grouped into five semantic categories. For each run, items are placed at randomized (x, y, θ) poses on the table to ensure environmental diversity.

Robot control is standardized across configurations. Each robot executes a grasp via 2D (x, y) picking coordinates in its local frame. A scripted grasping controller performs the following routine: (1) move the end-effector to a height of 15 cm above the target (x, y) position; (2) descend vertically until contact is detected via force sensing; (3) activate the suction gripper; and (4) lift the object back to the original height. This routine is fixed across all robots and trials. The controller and motion execution are handled by the CLIPort internal planner. The initial arm pose is fixed for all robots.

Network structure

Motion generator p_θ For robots with an image state (Pepper, Single-Arm, and the robots in the simulated item selection tasks), we first scale the image to size 224×224 and process it via three conv2D layers with batch normalization and max pooling operations. The output feature is then flattened and concatenated with a latent value z sampled from a unit Gaussian distribution. For robots with a vector state, we process the state via two linear layers with a size of 256 and concatenate the outputs with the sampled latent value z. For robots with both image and vector states, the concatenated feature is decoded via three linear layers with a size of 256 to generate the output action.

Motion encoder f_ψ For robots with an image state, we first scale the image to size 224×224, and then process and concatenate the output feature with the action using FiLM (feature wise linear modulation (*71*)) layers with 4 blocks. For robots with a vector state, we directly concatenate the state and the action. The concatenated features are then processed via three linear layers with a size of 256. The output is the intention encoding, which is a vector with a length of 256.

Annotation encoder f_ξ We follow the text encoder introduced in (*53*).

Network input and output

The input state to the encoder and the output action from the decoder for each robot used in the real-world experiments are presented in table S1, while those for the simulated tasks are summarized in table S2.

Training and evaluation data variations

For both real-world and simulated experiments, each trajectory is represented as a state-action pair (s, a), where action a transitions the robot from an initial state s to a goal state that completes a specific task. During testing, the predicted action a generated by our framework is executed via a robot-specific low-level controller. The state and action definitions for each robot are detailed in table S1 and table S2.

Real-world experiments We collected approximately 150 goal-state trajectories per robot to train the models, covering the full range of each robot's capabilities. The dataset spans diverse variations in monitoring locations, item identities, and object poses.

Cuboat performed monitoring at locations M_1 and M_2, collecting 75 samples per location. Tello performed monitoring at locations M_1–M_4, collecting 38 samples per location. Diablo performed monitoring at locations M_1–M_4, delivery to D_1–D_2, and item-receiving handovers from both

Table S1: **The state and action configurations of the seven robots used in the real-world scenario.** The x, y, and z coordinates of the Single-Arm and Dual-Arm robots represent the relative positions of their grippers with respect to the robot base. The x, y, and z coordinates and yaw of the remaining five robots indicate their global positions and rotations. The h, opening, picking, and item_status are special action signals defined for each robot. The values and explanations of these actions are provided below the table.

	Encoder Input (State)	Decoder Output (Action)
Cuboat	x,y, yaw	x,y, yaw
Tello	x,y,z, yaw	x,y,z, yaw
Diablo	x,y, yaw, h, item_status	x,y, yaw, h
Spark	x,y, yaw, item_status	x,y, yaw
Pepper	RGB image, item_status	x,y, yaw, picking
Single-Arm	RGB image (side & front camera)	x,y,z,opening
Dual-Arm	x,y,z,opening (left & right gripper)	x,y,z,opening (left & right gripper)

h: 0 - sit mode; 1 - stand mode. **opening**: 0 - close gripper; 1 - open gripper
picking: 0 - do not play picking animation; 1 - play picking animation.
item_status: 0 - item has not been picked; 1 - item picked by Pepper;
2 - item picked by Single-Arm; 3 - item picked by Dual-Arm

Table S2: **The state and action configurations of the robots used in the simulation tasks.** For the monitoring task, each robot's input and output match its control parameters. For the item picking task, the three UR5 variants receive an RGB image and output an (x, y) picking location. The camera for each UR5 robot is mounted at a different position (front, left shoulder, or right shoulder).

	Encoder Input (State)	Decoder Output (Action)
Pepper (Sim)	x, y, yaw, h_pitch	x, y, yaw, h_pitch
Wheeled Biped (Sim)	x, y, yaw, body_height	x, y, yaw, body_height
Drone (Sim)	x, y, z	x, y, z
Cater (Sim)	x, y, yaw	x, y, yaw
UR5_F, UR5_L, UR5_R (sim)	RGB image (front, left, right)	x, y

h_pitch: hip pitch angle of Pepper. **body_height**: height adjustment for Wheeled Biped.

Single-Arm and Dual-Arm, collecting 20 samples per task. Spark performed delivery to D_1–D_2 and item-receiving handovers from Single-Arm and Dual-Arm, collecting 38 samples per task. Pepper performed item picking for six items, delivery to D_1–D_2, and item-receiving handovers

from Single-Arm, collecting 90 samples for picking, 40 for delivery, and 20 for receiving. Single-Arm performed item picking for six items and handovers to Diablo and Pepper, collecting 90 samples for picking, 30 for handing over to Diablo, and 30 for handing over to Pepper. Dual-Arm performed item picking for six items and handovers to Diablo and Spark, collecting 90 samples for picking, 30 for handing over to Diablo, and 30 for handing over to Spark.

To evaluate real-world performance, we constructed 30 test scenarios featuring diverse team compositions and task configurations for both demonstrator and learner robots. Each demonstration scenario involved a fixed team of Tello, Dual-Arm, and Spark. For each case, the user location was uniformly sampled from four monitoring positions (M_1–M_4), an item was randomly selected from the six items available in the Dual-Arm's drawer, and the delivery destination was randomly chosen from two possible locations (D_1 or D_2).

In the corresponding learner scenarios, the team consisted of Cuboat, Pepper, Diablo, and Single-Arm. To simulate realistic variability in environmental resources, the available items for Pepper and Single-Arm were randomly sampled from each robot's item list. To further test the system's robustness, one learner robot was randomly removed in 10 of the 30 scenarios, creating partially incomplete team configurations.

Simulated experiments: For the two simulated tasks, including target monitoring and item selection, we collected 10,000 trajectories per robot for training, and an additional 500 trajectories per robot to serve as demonstrations during evaluation.

In the target monitoring task, we predefined a set of valid robot poses from which the target box was visible. The robot's (x, y) position is randomly sampled between 1.0 and 1.5 meters from the center of the table. Other embodiment-specific parameters, such as orientation (Cater), height (Wheeled Biped), and hip pitch angle (Pepper), are computed to center the target box in the robot's field of view. The state is defined as the robot's pose, and the action is the sampled target pose that preserves target visibility.

In the item selection task, at each trial the robot captured an RGB image from its onboard camera to represent the current state and then selected one of four items on the table as its target. The action is defined as the (x, y) location of the selected item's center in the robot's local coordinate frame. Objects are sampled from a predefined item list associated with each robot. Their positions and orientations are randomized at the start of each trial to ensure diverse training and evaluation distributions.

For both tasks, evaluation is conducted across all demonstrator–learner combinations, covering 16 robot pairs in the monitoring task and 9 robot pairs in the item selection task. For each robot pair, all 500 demonstrations are used during evaluation, with each learner attempting to reproduce the outcome of each demonstration. This procedure is repeated three times using models trained with different random seeds to assess robustness.

Hyperparameters

In the motion generator's training objective (Eq. (1)), we set the KL divergence weight β_{KL} to 0.01 divided by the dimensionality of the robot's action space. When generating batches of candidate actions (Eq. (11)), a fixed threshold of $\epsilon_{\mathrm{valid}} = 0.55$ is used to filter out out-of-distribution actions across both real-world and simulated tasks. Additionally, we define the alignment threshold $\epsilon_{\mathrm{aligned}}$, which determines whether a sampled action is considered a successful replication of a demonstration—as 0.335 for real-world tasks, 0.32 for the simulated item selection task, and 0.5 for the simulated monitoring task. These threshold values were selected based on tuning with 10 validation samples per robot that were held out from both training and evaluation.

In the IAIL framework, the motion generator for each robot is trained independently with a batch size of 128. The motion encoders and annotation encoders are trained jointly across all robots, using a batch size of 50 per robot. For the baseline methods, the density-based model employs a batch size of 128 for both the encoder and decoder, while the description-based model uses a batch size of 50 for training its motion and annotation encoders.

All models are optimized using the Adam optimizer with a learning rate of 1×10^{-4}. The IAIL motion generator and the encoder–decoder components of the density-based baseline are trained for 1,000 epochs, with 100 gradient updates per epoch. The motion and annotation encoders for both IAIL and the description-based baseline are trained for 250 epochs, also with 100 update steps per epoch.

Integrating Large Language Models

Our action association process is based on the intention representation, which are aligned with the natural language annotations. Therefore, our framework can be seamlessly integrated with LLMs to further extend its applicability with a wide range of tasks. In this section, we demonstrate how to utilize the planning capabilities of ChatGPT-4.0, a state-of-the-art LLM, to perform real-world composite tasks without relying on robot demonstration trajectories. We provide ChatGPT-4.0 with a text description of the task and request it to generate a set of language instructions for completing the task. Similar to the imitation of the trajectory demonstrations, our framework interprets the intentions underlying the text instructions, adapts them to executable actions for the most appropriate robot, and achieves outcomes similar to those detailed in the instructions.

fig. S4 illustrates an example of the task analysis, planning, and execution pipeline. fig. S4A shows the available robot team and items. fig. S4B shows the task description provided to the LLM. The output consists of a list of language instructions. fig. S4C depicts the output instructions and their execution by one of the robots on the team. The output instruction are limited to the annotations in the dataset, using a similar approach to that in (*72*). The video of this example is provided in the Supplementary Materials.

In the execution phase, three of the five robots completed the task based on instructions generated by ChatGPT-4.0. The outcomes closely mirror those obtained using robot trajectories as demonstrations, affirming that our framework can be effectively integrated into the pipeline of LLMs for task planning when robot trajectories are unavailable. This integration with LLMs notably enhances the framework's utility, allowing advancements in the rapidly evolving field of LLMs to be effectively leveraged with the proposed framework.

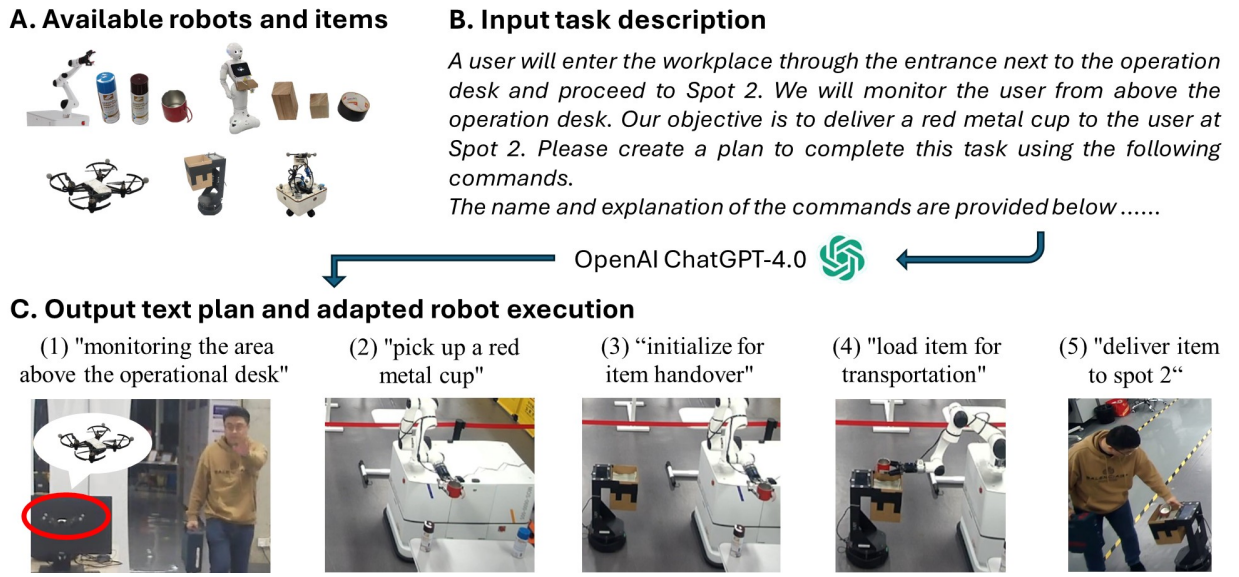

Figure S4: **The task planning and execution pipeline of IAIL integrated with ChatGPT-4.0.** (A) The available learner robot team and items. (B) The task descriptions given to ChatGPT-4.0. (C) The output plans and the execution results.

Supplementary Results

Visualization of latent intention representations

Fig. S5 presents a t-SNE visualization of the latent motion embeddings extracted from seven distinct robots involved in the real-world experiments. For each robot and each task type, we randomly selected three motion trajectories, resulting in a total of 120 samples. These trajectories comprehensively span the capabilities of the robots and were not included in training.

Each trajectory was processed by its corresponding robot-specific motion encoder, yielding a 256-dimensional latent vector. These vectors were then projected into a 2D space using the openTSNE implementation from scikit-learn and normalized to the range [0, 1] for visualization.

The resulting plot reveals distinct and coherent clusters, with each color indicating a different task and each marker shape representing a specific robot embodiment. The clear task-wise clustering

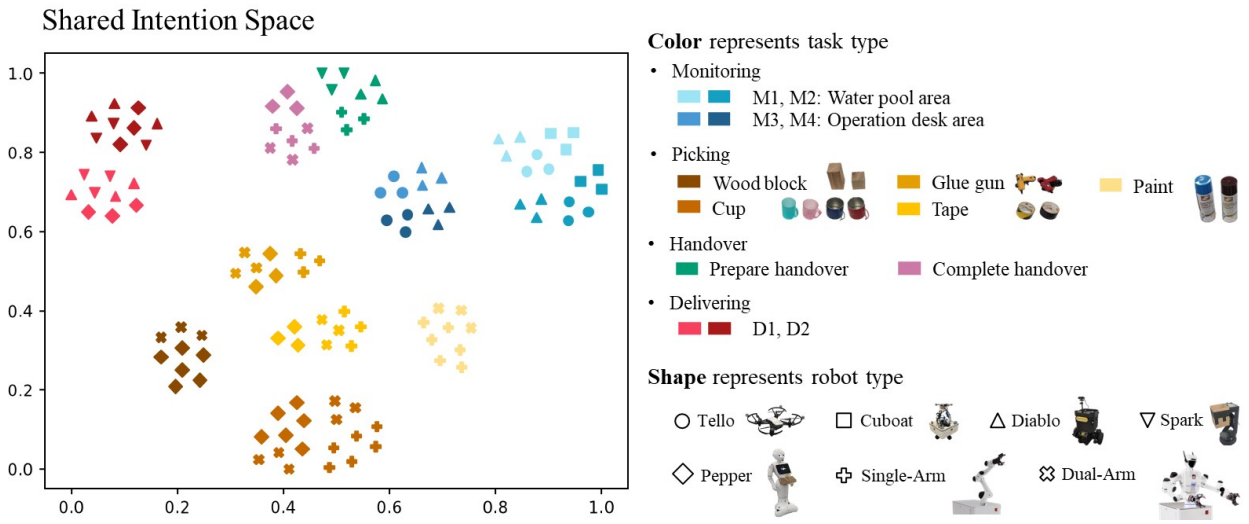

Figure S5: **t-SNE projection of 256-dimensional latent motion representations extracted by robot-specific encoders.** Each point represents a trajectory performed by one of seven robots, color-coded by task type and shaped by robot identity. The distinct clustering indicates strong semantic separation by task and effective cross-embodiment alignment of intentions.

and inter-robot alignment in the latent space demonstrate that the learned representation successfully captures both task semantics and embodiment-invariant intent.

Neighboring structure among phases

Beyond instance-level distinctions, the intention space exhibited coherent phase-level neighborhoods. As shown in the t-SNE projection (fig. S4), not only were individual tasks separated into distinct clusters, but tasks belonging to the same phase also occupied adjacent regions. For example, Picking variants, such as cup and tape, and Monitoring variants, such as M1 and M2, appeared in neighboring areas. To quantify this neighboring structure, we computed k-nearest-neighbor (k-NN) purity in the original intention-embedding space: for each sample, we retrieved its k nearest neighbors by cosine distance and measured the fraction sharing its phase label. We observed perfect local coherence for Monitoring, Handover, and Delivery (for k=10, mean 10-NN purity = 1.000 ± 0.000) and similarly high coherence for Picking (mean 10-NN purity = 0.972 ± 0.012). This high purity indicated a locally consistent geometry that preserved phase semantics and supported reliable intention association at inference time. Results were stable across $k \in \{5, 10, 15\}$. Together with the instance-level structure above, these coherent phase neighborhoods ensured that association preferentially retrieved motions that match both the phase and, when available, the task instance.

An analogous latent-space phenomenon was reported by (*73*), where joint training of language with motor/sensorimotor data induces structured latent representations that support generalization.

In their framework, a language-conditioned parametric-bias (PB) code learns a consistent verb–noun geometry that enables sentence-level compositional generalization in action generation. Our results paralleled this observation and extended it to a cross-embodiment setting. In our framework, we jointly trained a shared intention space with language annotations across heterogeneous embodiments; this latent space was embodiment-invariant and exhibited high local phase coherence as well as clear task separation. Whereas (*73*) employs PB to parameterize a single robot's predictive model and generate goal-consistent sequences, we instead leveraged latent distances at inference time to associate intentions and transfer motions across robots. Taken together, these results indicated that language-induced structured latent geometry was critical for robust motion generation; here we quantified this structure and demonstrated its utility for cross-embodiment association.

Summary tables for statistical tests

Table S3: **Pairwise test statistics comparing IAIL against the density-based baseline on the monitoring task.** The eight pairs with substantial distributional mismatch are marked in bold. Each test uses 1,500 samples per method. Reported statistics include the mean difference Δ (IAIL − baseline) with 95% CI, Welch's t statistic, degrees of freedom (df), two-sided p, and Cohen's d (95% CI). All values are shown with three decimals, p values smaller than 0.001 are shown as "< 0.001".

Learner – Demonstrator	Δ (95% CI)	t	df	p	Cohen's d (95% CI)
Pepper – Pepper	0.000 ([0.000, 0.000])	–	–	1.000	–
Pepper – Drone	0.101 ([0.068, 0.134])	6.032	2164.382	< 0.001	0.220 ([0.148, 0.292])
Pepper – Wheeled biped	0.855 ([0.820, 0.891])	47.273	2081.911	< 0.001	1.726 ([1.642, 1.810])
Pepper – Carter	1.000 ([1.000, 1.000])	–	–	< 0.001	–
Drone – Pepper	0.000 ([0.000, 0.000])	–	–	1.000	–
Drone – Drone	0.000 ([0.000, 0.000])	–	–	1.000	–
Drone – Wheeled biped	1.789 ([1.759, 1.820])	115.524	1499.000	< 0.001	4.218 ([4.090, 4.347])
Drone – Carter	2.000 ([2.000, 2.000])	–	–	< 0.001	–
Wheeled biped – Pepper	1.873 ([1.856, 1.891])	210.682	2956.896	< 0.001	7.693 ([7.486, 7.900])
Wheeled biped – Drone	1.761 ([1.729, 1.794])	106.827	1670.783	< 0.001	3.901 ([3.779, 4.023])
Wheeled biped – Wheeled biped	−0.002 ([-0.004, 0.000])	−1.733	1499.000	0.083	−0.063 ([-0.135, 0.008])
Wheeled biped – Carter	0.000 ([0.000, 0.000])	–	–	1.000	–
Carter – Pepper	1.000 ([1.000, 1.000])	–	–	< 0.001	–
Carter – Drone	0.900 ([0.867, 0.933])	52.898	2185.058	< 0.001	1.932 ([1.845, 2.018])
Carter – Wheeled biped	0.125 ([0.090, 0.159])	7.126	2102.713	< 0.001	0.260 ([0.188, 0.332])
Carter – Carter	0.000 ([0.000, 0.000])	–	–	1.000	–

For learner–demonstrator pairs with nearly deterministic scores for both methods (extremely small within-condition SDs), the t, df, p, and Cohen's d become ill-defined or numerically unstable. For such rows, we report "–" for t, df and Cohen's d, and instead report a two-sided permutation p value.

Table S4: **Pairwise test statistics comparing IAIL against the description-based baseline on the monitoring task.** The four pairs with capability mismatch are marked in bold. Each test uses 1,500 samples per method. Reported statistics include the mean difference Δ (IAIL − baseline) with 95% CI, Welch's t statistic, degrees of freedom (df), two-sided p, and Cohen's d (95% CI). All values are shown with three decimals, and p values smaller than 0.001 are shown as "< 0.001".

Learner – Demonstrator	Δ (95% CI)	t	df	p	Cohen's d (95% CI)
Pepper – Pepper	0.000 ([0.000, 0.000])	–	–	1.000	–
Pepper – Drone	0.080 ([0.048, 0.112])	4.972	2230.989	< 0.001	0.182 ([0.110, 0.253])
Pepper – Wheeled biped	0.875 ([0.841, 0.910])	49.992	2125.868	< 0.001	1.825 ([1.740, 1.911])
Pepper – Carter	1.000 ([1.000, 1.000])	–	–	< 0.001	–
Drone – Pepper	0.000 ([0.000, 0.000])	–	–	1.000	–
Drone – Drone	0.000 ([0.000, 0.000])	–	–	1.000	–
Drone – Wheeled biped	0.000 ([0.000, 0.000])	–	–	1.000	–
Drone – Carter	0.000 ([0.000, 0.000])	–	–	1.000	–
Wheeled biped – Pepper	−0.055 ([−0.067, −0.044])	−9.370	1499.000	< 0.001	−0.342 ([−0.414, −0.270])
Wheeled biped – Drone	−0.023 ([−0.030, −0.015])	−5.896	1499.000	< 0.001	−0.215 ([−0.287, −0.144])
Wheeled biped – Wheeled biped	−0.002 ([−0.004, 0.000])	−1.733	1499.000	0.083	−0.063 ([−0.135, 0.008])
Wheeled biped – Carter	0.000 ([0.000, 0.000])	–	–	1.000	–
Carter – Pepper	1.000 ([1.000, 1.000])	–	–	< 0.001	–
Carter – Drone	0.880 ([0.845, 0.915])	49.922	2133.334	< 0.001	1.823 ([1.738, 1.908])
Carter – Wheeled biped	0.094 ([0.062, 0.126])	5.687	2183.600	< 0.001	0.208 ([0.136, 0.279])
Carter – Carter	0.000 ([0.000, 0.000])	–	–	1.000	–

For learner–demonstrator pairs with nearly deterministic scores for both methods (extremely small within-condition SDs), the t, df, p, and $Cohen'sd$ become ill-defined or numerically unstable. For such rows, we report "–" for t, df and Cohen's d, and instead report a two-sided permutation p value.

Table S5: **Pairwise test statistics comparing IAIL against the density-based baseline on the item picking task.** Each test uses 1,500 samples per method. Reported statistics include the mean difference Δ (IAIL − baseline) with 95% CI, Welch's t statistic, degrees of freedom (df), two-sided p, and Cohen's d (95% CI). p values smaller than 0.001 are shown as "< 0.001".

Learner – Demonstrator	Δ (95% CI)	t	df	p	Cohen's d (95% CI)
UR5_L – UR5_L	1.129 ([1.085, 1.173])	50.408	2877.457	< 0.001	1.841 ([1.755, 1.926])
UR5_L – UR5_F	1.077 ([1.040, 1.113])	57.847	2520.248	< 0.001	2.112 ([2.023, 2.202])
UR5_L – UR5_R	1.023 ([0.983, 1.064])	49.466	2814.127	< 0.001	1.806 ([1.721, 1.891])
UR5_F – UR5_L	1.085 ([1.047, 1.123])	56.506	2696.794	< 0.001	2.063 ([1.975, 2.152])
UR5_F – UR5_F	1.149 ([1.105, 1.193])	51.469	2941.928	< 0.001	1.879 ([1.793, 1.965])
UR5_F – UR5_R	1.104 ([1.068, 1.140])	60.366	2618.048	< 0.001	2.204 ([2.114, 2.295])
UR5_R – UR5_L	1.122 ([1.085, 1.159])	59.185	2553.664	< 0.001	2.161 ([2.071, 2.251])
UR5_R – UR5_F	1.105 ([1.066, 1.143])	56.671	2583.603	< 0.001	2.069 ([1.981, 2.158])
UR5_R – UR5_R	1.183 ([1.137, 1.229])	50.456	2940.506	< 0.001	1.842 ([1.757, 1.928])

Table S6: **Pairwise test statistics comparing IAIL against the description-based baseline on the item picking task.** Each test uses 1,500 samples per method. Reported statistics include the mean difference Δ (IAIL − baseline) with 95% CI, Welch's t statistic, degrees of freedom (df), two-sided p, and Cohen's d (95% CI). p values smaller than 0.001 are shown as "< 0.001".

Learner – Demonstrator	Δ (95% CI)	t	df	p	Cohen's d (95% CI)
UR5_L – UR5_L	0.669 ([0.613, 0.725])	23.454	2379.825	< 0.001	0.856 ([0.782, 0.931])
UR5_L – UR5_F	0.742 ([0.697, 0.787])	32.025	2135.223	< 0.001	1.169 ([1.092, 1.247])
UR5_L – UR5_R	0.577 ([0.527, 0.628])	22.309	2351.528	< 0.001	0.815 ([0.740, 0.889])
UR5_F – UR5_L	0.709 ([0.662, 0.756])	29.661	2261.337	< 0.001	1.083 ([1.006, 1.160])
UR5_F – UR5_F	0.684 ([0.628, 0.739])	24.034	2457.242	< 0.001	0.878 ([0.803, 0.953])
UR5_F – UR5_R	0.740 ([0.694, 0.787])	31.174	2141.035	< 0.001	1.138 ([1.061, 1.215])
UR5_R – UR5_L	0.514 ([0.466, 0.562])	20.992	2107.229	< 0.001	0.767 ([0.692, 0.841])
UR5_R – UR5_F	0.525 ([0.477, 0.573])	21.490	2168.786	< 0.001	0.785 ([0.710, 0.859])
UR5_R – UR5_R	0.466 ([0.407, 0.524])	15.609	2457.402	< 0.001	0.570 ([0.497, 0.643])

Caption for Movie S1. Animated version of Figure 3.

Caption for Movie S2. Animated version of Figure 2.

Caption for Movie S3. Animated version of Figure S4.